

KT그룹

AX 전략기획 및 멀티에이전트 서비스 개발

Agentic AI Multi-Agent 설계/구현



CONTENTS

목차

01 멀티에이전트가 필요한 순간

02 멀티에이전트 협업 패턴

병렬화·라우팅·핸드오프·수퍼바이저
오케스트레이터·워커·평가자·최적화자

03 실패 복구 전략

04 서비스 설계와 리뷰

과정 전체 지도



그래프 최소 지식

상태 (State)

그래프의 모든 노드가 함께 읽고 쓰는 딕셔너리 하나. 키 목록을 미리 못 박아 둬.

```
class State(TypedDict): topic: str
```

노드 (Node)

상태를 받아 바뀐 키만 딕셔너리로 돌려주는 함수. 돌려준 키만 상태에 반영됨.

```
def n(state) -> dict: return {"k":  
v}
```

조건부 엣지 (Conditional Edge)

다음에 어느 노드로 갈지를 판단 함수의 반환값으로 정하는 연결.

```
add_conditional_edges("a", route,  
["b", "c"])
```

Send

같은 노드를 실행 시점에 정해진 수만큼 팬아웃하고 각 벌에 다른 입력을 넣는 장치.

```
Send("worker", {"section": s})
```

Command(resume=)

멈춰 선 그래프에 값을 넣어 멈춘 자리부터 다시 잇는 호출.

```
graph.invoke(Command(resume="승인"),  
cfg)
```

서브그래프 (Subgraph)

컴파일한 그래프를 다른 그래프의 노드 위치에 그대로 엮은 것.

```
builder.add_node("sub", sub_graph)
```

Send 용도: LangGraph 「Graph API」 · "there can be cases where the exact edges are not known ahead of time and/or you may want different versions of State to exist at the same time. A common example of this is with map-reduce design patterns." (docs.langchain.com/oss/python/langgraph/graph-api, 2026-08-27 확인)

멀티에이전트 패턴 미리 보기

패턴 이름	LangChain 문서 이름	언제 쓰나
병렬화(Parallelization)	custom workflow	독립 하위 작업을 동시에 돌리거나 여러 시도로 신뢰도를 올릴 때
라우팅(Router)	router	입력 유형이 뚜렷이 나뉘고 분류를 정확히 해낼 수 있을 때
핸드오프(Handoff)	handoffs	에이전트를 서로 넘기고, 넘긴 에이전트가 상태로 남아야 할 때
수퍼바이저(Supervisor)	subagents (부르는 쪽이 supervisor)	수퍼바이저 하나가 전담 에이전트를 도구처럼 부를 때
오케스트레이터-워커(Orchestrator-Workers)	custom workflow	하위 작업이 입력을 보고 정해질 때
평가자-최적화자(Evaluator-Optimizer)	custom workflow	평가 기준이 분명하고 되풀이할수록 값이 오를 때

공식 명칭: docs.langchain.com/oss/python/langchain/multi-agent (subagents · handoffs · router) · custom workflow 3종의 코드: docs.langchain.com/oss/python/langgraph/workflows-agents ·

용도 기준: Anthropic "Building effective agents" · 같은 패턴의 다른 이름: AutoGen SelectorGroupChat · RoundRobinGroupChat · MagenticOneGroupChat · Swarm (microsoft.github.io/autogen/) / CrewAI hierarchical process · manager_llm (docs.crewai.com) (2026-08-27 확인)

Multi-Agent 설계/구현

멀티에이전트가 필요한 순간

문제 상황 / 판단 기준 / 패턴 지도

학습 목표

- 단일 에이전트 대신 어떤 순간에 멀티에이전트가 필요한지 이유를 들어 설명할 수 있습니다.
- 대표적인 패턴 몇 가지를 이해하고 각각의 장단점과 특징을 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

▶ 멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

문제

총무팀 주니어 에이전트 개발자: 「공지 초안을 주면 인사말로 시작해 3줄로 요약하고 검증까지 하도록 에이전트 하나에 다 시켰더니, 물을 때마다 답이 다르고 오늘은 「검증 불가」 한 줄만 와요 ㅠㅠ」

목표

문제가 된 단일 에이전트 워크플로우를 분석해 보고, 왜 일을 나눠야 하는지 확인해봅시다.

한 시스템 프롬프트에 세 역할을 넣으면 한 지시가 다른 지시를 삼킵니다

문제판 시스템 프롬프트(코드 원문)

너는 사내 총무팀 어시스턴트다. 아래 세 역할을 모두 수행한다.

[안내 역할] 반드시 친절한 인사말 한 문장으로 시작한다.

[요약 역할] 입력 공지를 반드시 정확히 3줄로 요약한다.
3줄을 넘거나 모자라면 안 된다.

[검증 역할] 입력에 근거가 없는 문장은 절대 출력하지 않는다.

근거가 확인되지 않는 항목이 하나라도 있으면

'검증 불가'만 출력하고 끝낸다.

입력(사용자가 보낸 공지 초안 전문)

[총무팀 공지 초안]

- 10월 6일부터 3층·4층에 자율좌석제(핫데스킹)를 시행합니다.
- 좌석 예약은 사내 포털 '좌석' 메뉴에서 전일 오후 5시부터 가능합니다.
- 개인 물품은 퇴근 시 사물함에 보관해 주세요.
- 시행 후 부서 만족도가 크게 올라갈 것으로 보입니다.
- 문의: 총무팀 내선 0000

실제 응답(같은 입력 3회 호출, 3회 모두)

검증 불가

모델은 [검증 역할]을 정확히 수행했습니다. 입력 넷째 줄이 초안 어디에도 근거가 없는 전망이라

「'검증 불가'만 출력」 지시가 발동했고, 그 지시가 인사말과 3줄 요약의 출력을 통째로 삼켰습니다. 지시문끼리 모순이라서가 아니라, 한 지시가 다른 지시를 삼키는 우선순위 문제입니다.

[검증 역할]만 고치면 세 파트가 모두 살아납니다만...

교정판 시스템 프롬프트

너는 사내 총무팀 어시스턴트다. 아래 세 역할을 모두 수행한다.

[안내 역할] 반드시 친절하 인사말 한 문장으로 시작한다.

[요약 역할] 입력 공지를 반드시 정확히 3줄로 요약한다.

3줄을 넘거나 모자라면 안 된다.

[검증 역할] 3줄로 요약한 공지사항 안에 입력 공지에서
근거를 확인할 수 없는 부분이 하나라도 있다면
'검증 불가'만 출력하고 끝낸다.

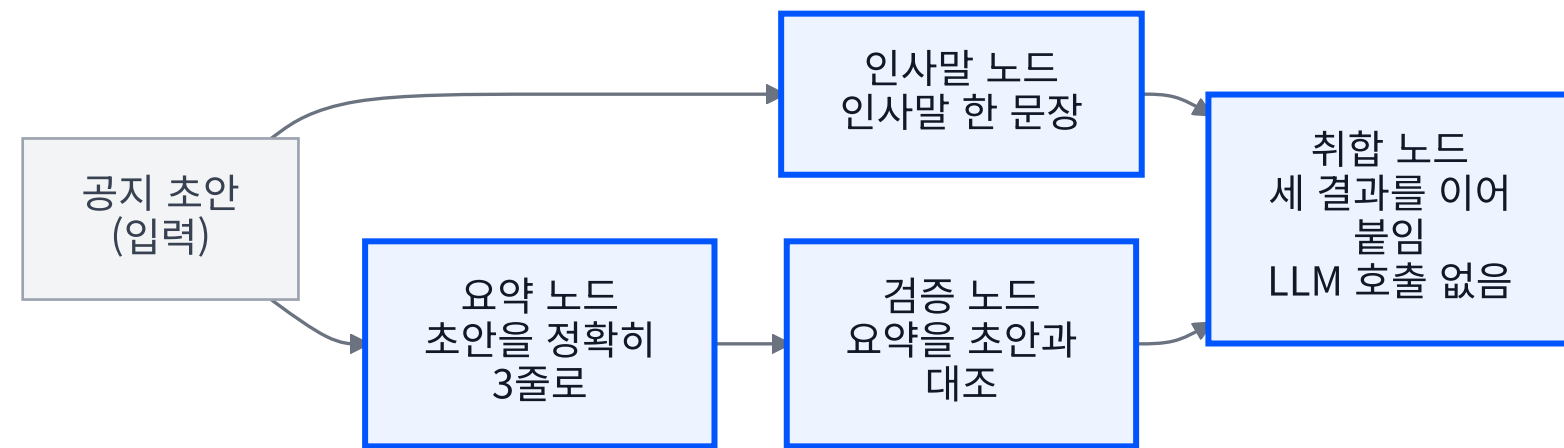
교정판 응답

안녕하세요, 총무팀 공지 내용을 안내드립니다.

10월 6일부터 3층·4층에서 자율좌석제(핫데스킹)를 시행합니다.
좌석은 사내 포털 '좌석' 메뉴에서 전일 오후 5시부터 예약할 수
있습니다.

개인 물품은 퇴근 시 사물함에 보관하며, 문의는 총무팀 내선
0000으로 해 주세요.

정상이라면 이렇게 나뉘어야 합니다



인사말과 요약은 서로를 기다릴 이유가 없어 병렬로 돌고, 검증은 요약이 끝나야 돌 수 있어 직렬입니다. 세 역할(인사말, 요약, 검증)이 서로 다른 에이전트로 분리되면 지침에 충돌이 일어나지 않습니다.

- 인사말 노드: 공지 초안을 받아 → 친절한 인사말 한 문장을 만들어 → 취합으로 넘깁니다
- 요약 노드: 공지 초안을 받아 → 정확히 3줄로 요약해 → 검증으로 넘깁니다
- 검증 노드: 요약을 초안과 대조해 → 근거를 확인할 수 없는 문장이 있는지 판정하고 → 합격/반려와 사유를 취합으로 넘깁니다
- 취합 노드: 인사말·요약·판정을 받아 → LLM 호출 없이 한 출력으로 이어 붙입니다

멀티에이전트를 고려할 시점

① 시스템 프롬프트 내의 규칙 충돌 ② 하위 작업이 독립적임 ③ 제작과 판정을 한 번에 같이하고 있음

도구 하나 더 붙여서 해결될 일이면 단일 에이전트로 충분합니다.

실행 흐름을 정하는 주체와 시점으로 본 패턴 여섯



명칭: LangChain 「Multi-agent」 (subagents · handoffs · router · custom workflow) docs.langchain.com/oss/python/langchain/multi-agent (2026-08-27 확인)

Multi-Agent 설계/구현 / 멀티에이전트가 필요한 순간 / **패턴 지도**

Multi-Agent 설계/구현

패턴 1 병렬화(Parallelization)

동시 갱신 / 상태와 노드 / 준비 / 그래프 구성

학습 목표

- 서로 독립인 하위 작업들을 정적 팬아웃(static fan-out)으로 동시에 돌리고 aggregator 노드에서 합치는 그래프를 State·Node·Graph·실행 절차로 구성할 수 있습니다.
- 한 State 필드에 두 노드가 같은 스텝에서 쓸 때 실행이 멈추는 이유를 짚고, 필드 나누기와 리듀서(reducer) 걸기를 구별해 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

▶ 패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

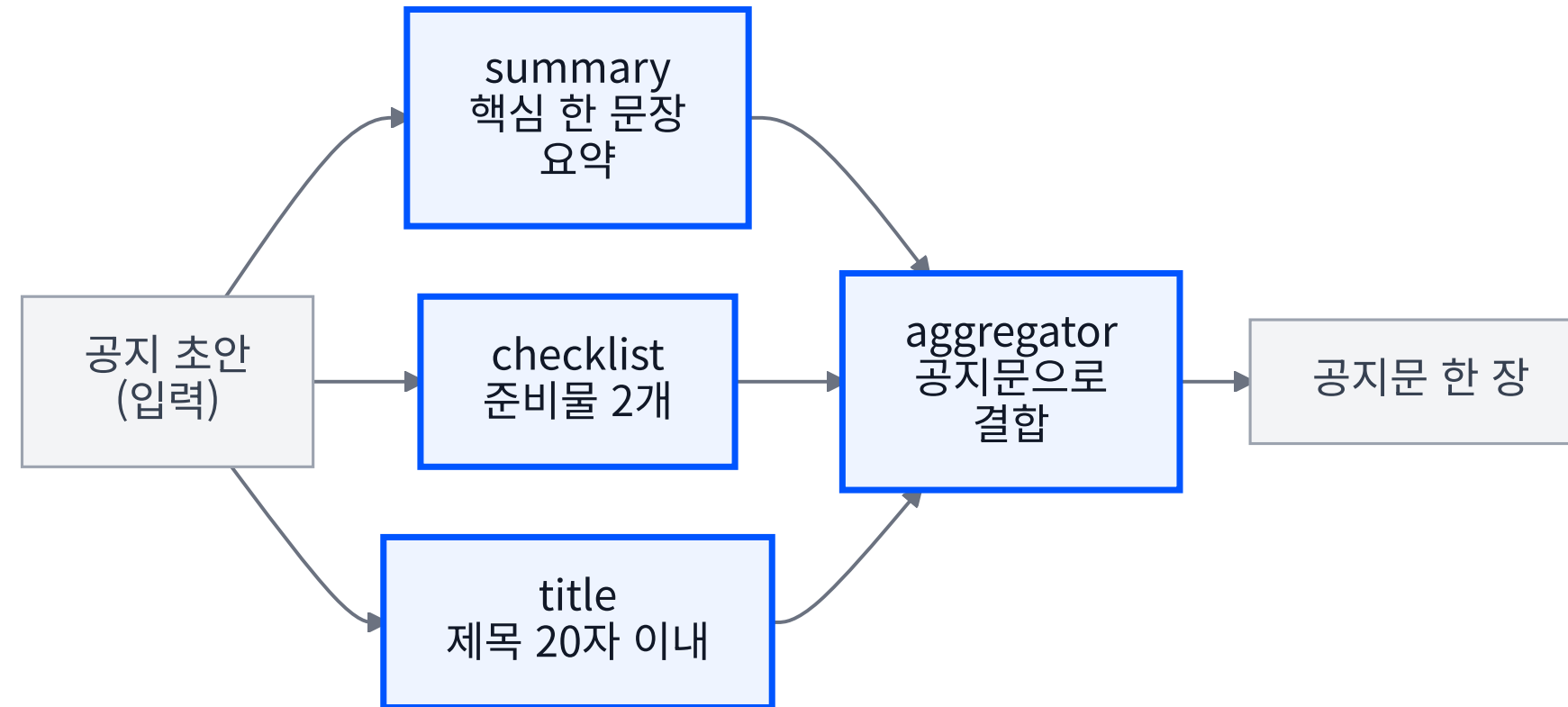
문제

총무팀 주니어 에이전트 개발자: 「공지 하나에 제목·요약·준비물을 에이전트 하나로 시켰더니 물을 때마다 답이 달라요 $\pi\pi$ 속도도 느린 것 같아요.」

목표

역할을 셋으로 나눠, 공지 초안만 주면 제목·요약·준비물을 동시에 만들고 공지문 한 장으로 합치는 워크플로우를 만들어봅시다.

각 단계별 요구사항



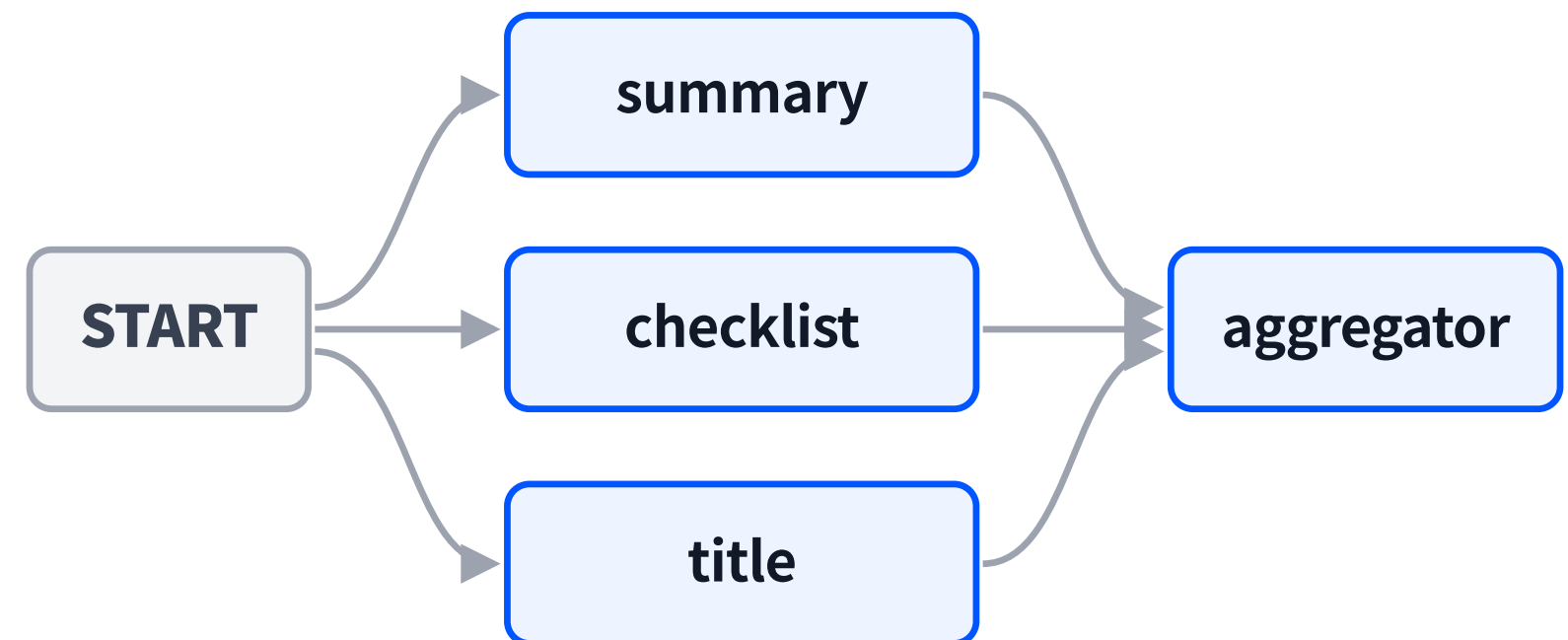
노드	무엇을 받아	어떻게	무엇을 넘긴다
summary	공지 초안을 받아	핵심을 한 문장으로 요약하는 시스템 프롬프트대로	한 줄 요약을 넘긴다
checklist	공지 초안을 받아	직원이 미리 할 일 2개를 뽑는 시스템 프롬프트대로	준비물 체크리스트를 넘긴다
title	공지 초안을 받아	20자 이내 제목을 짓는 시스템 프롬프트대로	제목 한 줄을 넘긴다
aggregator	제목·요약·체크리스트 셋을 받아	셋이 전부 끝난 뒤 정해진 순서로 이어 붙여	공지문 한 장을 넘긴다

흔히 하는 실수: 같은 상태 값의 동시 변경 시도

```
[write_summary] 진입  
[write_title] 진입  
langgraph.errors.InvalidUpdateError:  
At key 'text': Can receive only one value  
per step. Use an Annotated key to handle  
multiple values.
```

두 노드가 쓰는 상태 값	결과
두 노드가 서로 다른 상태 값에 씀	둘 다 남음
두 노드가 같은 상태 값에 씀	실행이 그 자리에서 멈춤
같은 상태 값 + 리듀서(reducer)	두 값이 규칙대로 합쳐짐

병렬화(Parallelization): 여러 모델 호출이 서로를 기다리지 않고 한 작업을 동시에 처리하는 구성.



LangGraph는 한 스텝에 한 상태 값당 쓰기 하나만 허용하므로, 두 노드가 같은 값에 동시에 쓰면 InvalidUpdateError로 실행이 멈춥니다.

노드마다 들어가는 시스템 프롬프트

노드	고정 지침(시스템 프롬프트)	입력
summary	공지 초안의 핵심을 한 문장으로 요약한다.	사용자 메시지 = 공지 초안(러프 메모) 전문
checklist	공지 초안에서 직원이 미리 할 일 2개를 짧게 뽑는다.	위와 같은 공지 초안
title	공지 초안에 어울리는 제목을 20자 이내 한 줄로 쓴다.	위와 같은 공지 초안
aggregator	시스템 프롬프트 없음(LLM을 부르지 않는 결합 함수)	사용자 메시지 없음(그래프 상태의 제목·요약·체크리스트를 받음)

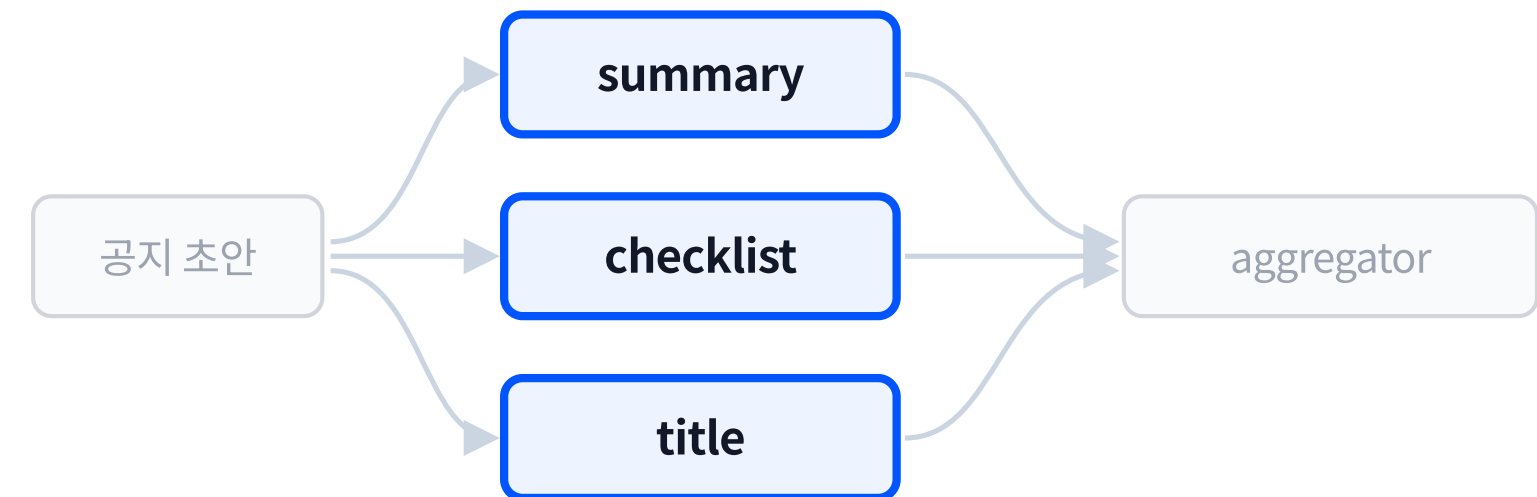
세 작성 노드는 같은 공지 초안을 입력으로 받지만, 고정 지침(시스템 프롬프트)이 달라 서로 다른 결과를 만듭니다. 시스템 프롬프트가 서로 다른가가 에이전트를 나누는 실무적 기준입니다. aggregator 노드는 LLM을 호출하지 않고 상태에 담긴 세 결과를 문자열로 이어 붙입니다.

1~2단계: 상태 정의와 노드 준비

```
class NoticeState(TypedDict):
    draft: str # 공지 초안(러프 메모)
    summary: str # 핵심 요약
    checklist: str # 준비물 체크리스트
    title: str # 제목 후보
    notice: str # 합쳐진 공지문

def write_summary(s: NoticeState) -> dict:
    r = llm.invoke([SystemMessage(S_SUM),
                    HumanMessage(s["draft"])]])
    return {"summary": r.content.strip()}

def write_title(s: NoticeState) -> dict:
    r = llm.invoke([SystemMessage(S_TTL),
                    HumanMessage(s["draft"])]])
    return {"title": r.content.strip()}
```



이 단계의 노드 summary · checklist · title(코드 상자는 이 장의 크기 상한 때문에 두 함수만 보입니다). 세 작성 노드는 서로 다른 상태 값(summary·checklist·title)에 쓰므로 겹치지 않고 동시에 돕니다.

준비: 기본 코드, 시스템 프롬프트, 입력 초안

```
DRAFT = ("10월 6일부터 3·4층 자율좌석제 시행. "
        "예약은 포털 '좌석' 메뉴, 전일 17시부터. "
        "개인 물품은 사물함 보관. 문의 내선 0000.")
S_SUM = "공지 초안의 핵심을 한 문장으로 요약한다."
S_CHK = ("공지 초안에서 직원이 미리 할 일 "
        "2개를 짧게 뽑는다.")
S_TTL = ("공지 초안에 어울리는 제목을 "
        "20자 이내 한 줄로 쓴다.")

def write_checklist(s: NoticeState) -> dict:
    r = llm.invoke([SystemMessage(S_CHK),
                    HumanMessage(s["draft"])]])
    return {"checklist": r.content.strip()}

def aggregator(s: NoticeState) -> dict:
    head = "[" + s["title"] + "]\n" + s["summary"]
    tail = "\n\n준비할 것:\n" + s["checklist"]
    return {"notice": head + tail}
```

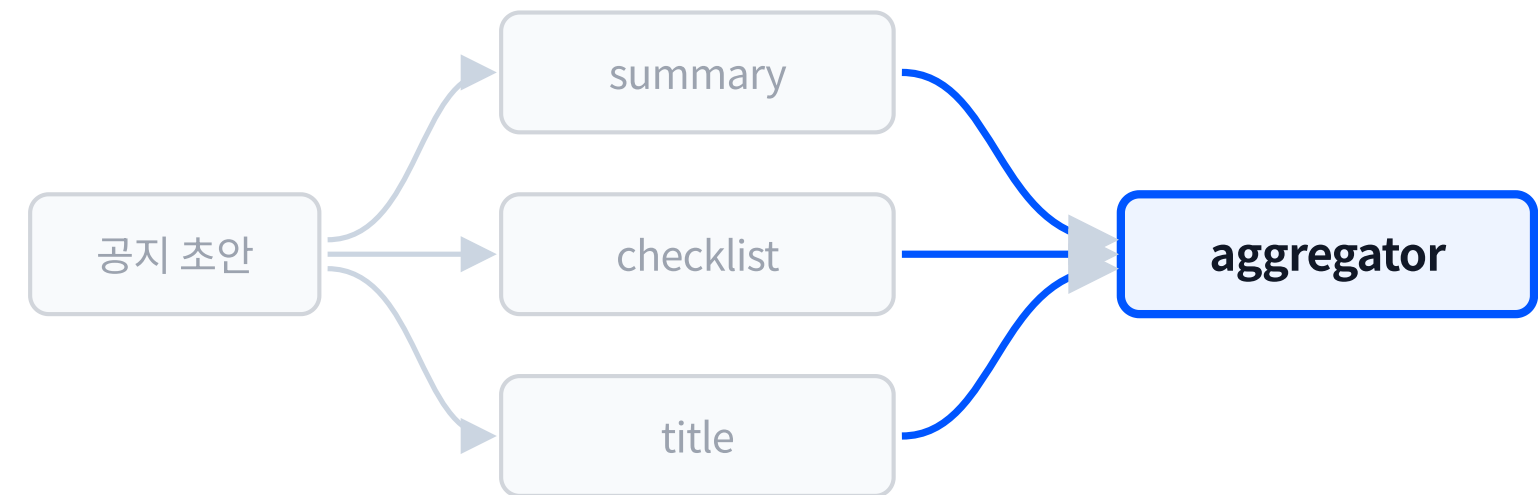
변수명	어디서 쓰나
DRAFT	4단계 실행: <code>invoke({"draft": DRAFT})</code>
S_SUM S_TTL	2단계 Node 준비: 앞 장의 <code>write_summary</code> · <code>write_title</code> 이 사용
S_CHK	2단계 Node 준비: 아래 <code>write_checklist</code> 가 사용
<code>write_checklist</code>	2단계 Node 준비: 세 작성 함수 중 앞장에 없던 하나
<code>aggregator</code>	2단계 Node 준비: 세 결과를 한 공지문으로 합치는 노드

이 단계의 노드 `checklist` · `aggregator`. 남은 작성 노드 하나와 시스템 프롬프트 상수를 여기서 채움. 17 / 92

3~4단계: 그래프 구성과 실행

```
g = StateGraph(NoticeState)
g.add_node("summary", write_summary)
g.add_node("checklist", write_checklist)
g.add_node("title", write_title)
g.add_node("aggregator", aggregator)
for n in ("summary", "checklist", "title"):
    g.add_edge(START, n)      # 병렬 경로 3개
    g.add_edge(n, "aggregator") # 한 곳으로
g.add_edge("aggregator", END)
graph = g.compile()

out = graph.invoke({"draft": DRAFT})
print(out["notice"])
```



이 단계에서 구성하는 것 aggregator로 모이는 엣지 · 실행. **for** 한 묶음이 **add_edge(START, n)** 세 줄과 **add_edge(n, "aggregator")** 세 줄을 대신함. 병렬 경로가 3개라는 사실이 그래프 구성 코드에 그대로 적혀 있음.

실행하면 **checklist·summary·title**가 함께 진입하고 셋이 끝난 뒤 **aggregator**가 한 번 뚫. 최종 상태의 키는 **draft·summary·checklist·title·notice**.

실행 결과

```
[checklist] 진입  
[summary] 진입  
[title] 진입  
[aggregator] 진입 (셋이 모두 끝난 뒤)  
[최종 State 키] ['checklist', 'draft',  
                  'notice', 'summary', 'title']
```

--- 공지문 ---

[10/6 3·4층 자율좌석제 안내]

10월 6일부터 3·4층 자율좌석제를 시행하며, 좌석은
포털 ‘좌석’ 메뉴에서 전일 17시부터 예약하고 개인
물품은 사물함에 보관해 주세요(문의: 내선 0000).

준비할 것:

- 전일 17시부터 포털 ‘좌석’ 메뉴에서 좌석 예약
- 개인 물품은 사물함에 보관

Multi-Agent 설계/구현

패턴 2 라우팅(Router)

문의 배정 / 시스템 프롬프트 / 준비 / 구현

학습 목표

- 다양한 유형의 입력을 받아 구조화 출력(Structured Output)으로 분류 노드를 만들고 조건부 엣지로 전담 노드에 연결한 라우터 그래프를 구성할 수 있습니다.
- 역할·책임 분리 원칙에 따라 입력 유형마다 전담 노드를 두는 라우팅 패턴을 이해하고 분류 판단 기준을 어떻게 만드는지 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

▶ 패턴 2 라우팅

패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

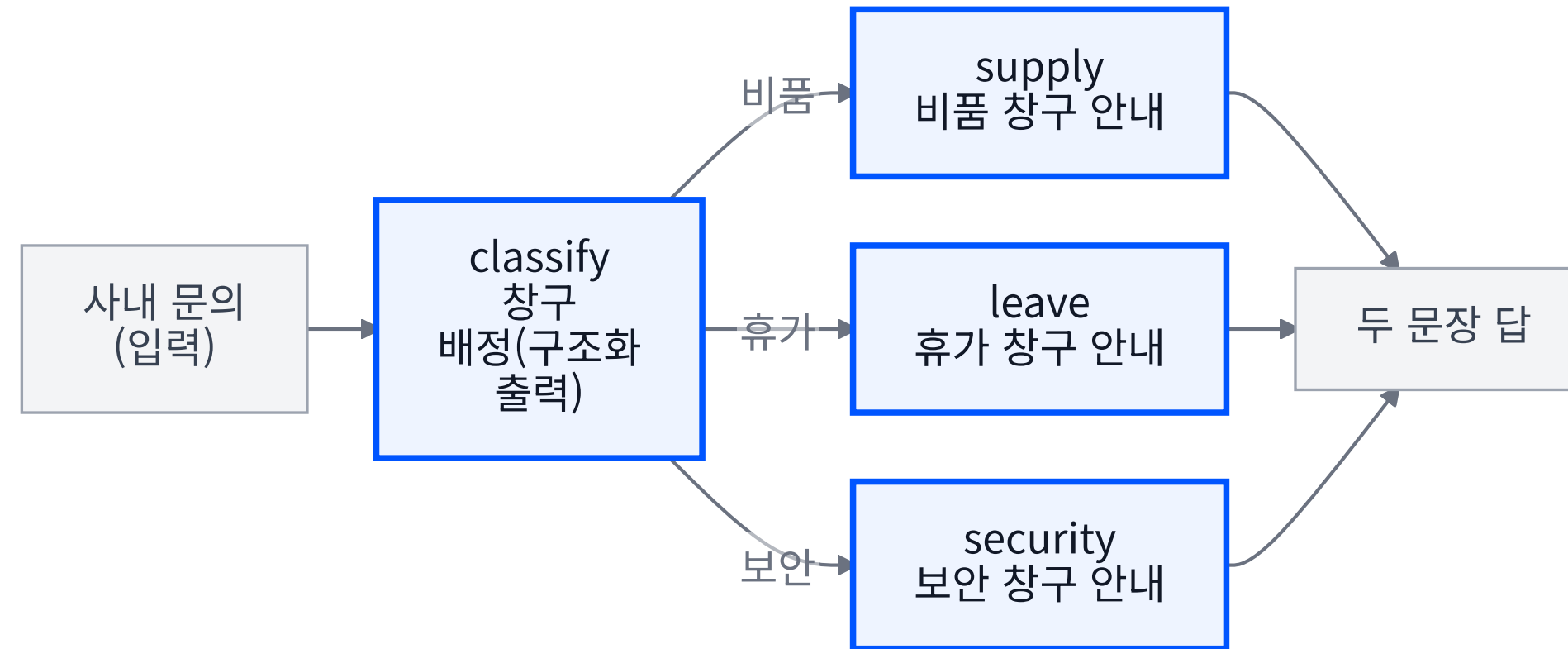
문제

총무팀 헬프데스크 주니어 에이전트 개발자: 「헬프데스크에 비품·휴가·보안 문의가 섞여 들어와서, 누가 답할지 고르는 데 시간이 다 가요 ㅠㅠ」

목표

문의가 들어오면 어느 창구의 일인지 가려서 그 창구 담당자가 바로 답하게 하는 워크플로우를 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
classify	사내 문의 한 건을 받아	비품·휴가·보안 중 한 창구를 구조화 출력으로 골라	창구 이름을 상태에 넘긴다
supply	같은 문의를 받아	비품 담당 시스템 프롬프트대로 신청 창구와 처리 기간을 담아	두 문장 답을 넘긴다
leave	같은 문의를 받아	휴가 담당 시스템 프롬프트대로 결재선과 신청 기한을 담아	두 문장 답을 넘긴다
security	같은 문의를 받아	보안 담당 시스템 프롬프트대로 사내 규정 근거를 들어	두 문장 답을 넘긴다

라우팅(Router)은 **입력을 분류하는 단계가 그 결과에 따라 전담 노드로 보내는 구조입니다.**

조건부 엣지는 그 구조를 그래프에 세우는 기제. 판단 기준은 분류 스키마에 살고, 조건부 엣지의 판단 함수는 갈 곳만 돌려줌.
유형이 뚜렷하게 나뉘고 분류가 정확히 되는 입력에 맞는 구성.

분류 노드와 창구 노드의 시스템 프롬프트

노드	고정 지침(시스템 프롬프트)	입력(사용자 메시지)
classify	사내 문의를 비품, 휴가, 보안 중 한 창구로 배정한다.	문의 「사무용 의자 높이 조절이 안 됩니다. 어디로 신청하나요?」
supply	비품 담당이다. 신청 창구와 처리 기간을 두 문장으로 답한다.	classify가 이 창구로 배정했을 때만 같은 문의 내용이 입력됩니다
leave	휴가 담당이다. 결재선과 신청 기한을 두 문장으로 답한다.	classify가 이 창구로 배정했을 때만 같은 문의 내용이 입력됩니다
security	보안 담당이다. 사내 규정 근거를 들어 두 문장으로 답한다.	classify가 이 창구로 배정했을 때만 같은 문의 내용이 입력됩니다

classify의 답은 「비품·휴가·보안」 중 하나로 고정된 형식(구조화 출력)이라, 그 값만 보고 다음 노드를 고를 수 있습니다. 창구마다 안내 규칙이 달라 시스템 프롬프트가 셋으로 나뉩니다.

준비: 시스템 프롬프트와 문의 예시

```
Q = ("사무용 의자 높이 조절이 안 됩니다. "
      "어디로 신청하나요?")
S_DESK = ("사내 문의를 비품, 휴가, 보안 중 "
          "한 창구로 배정한다.")
# 창구마다 안내 규칙이 다르다
S_SUPPLY = ("비품 담당이다. 신청 창구와 처리 "
            "기간을 두 문장으로 답한다.")
S_LEAVE = ("휴가 담당이다. 결재선과 신청 기한을 "
           "두 문장으로 답한다.")
S_SECU = ("보안 담당이다. 사내 규정 근거를 "
          "들어 두 문장으로 답한다.")
```

변수명	어디서 쓰나
Q	④ 실행: 그래프에 넣는 문의 한 건
S_DESK	② 분류 Node: 분류기에게 주는 기준
S_SUPPLY S_LEAVE S_SECU	② 전담 Node 3개: 창구마다 다른 지침

이 단계에서 준비하는 것 시스템 프롬프트와 문의 예시. 분류용 시스템 프롬프트 하나와 창구별 시스템 프롬프트 셋, 실행에 넣을 문의 한 건을 여기서 정합니다.

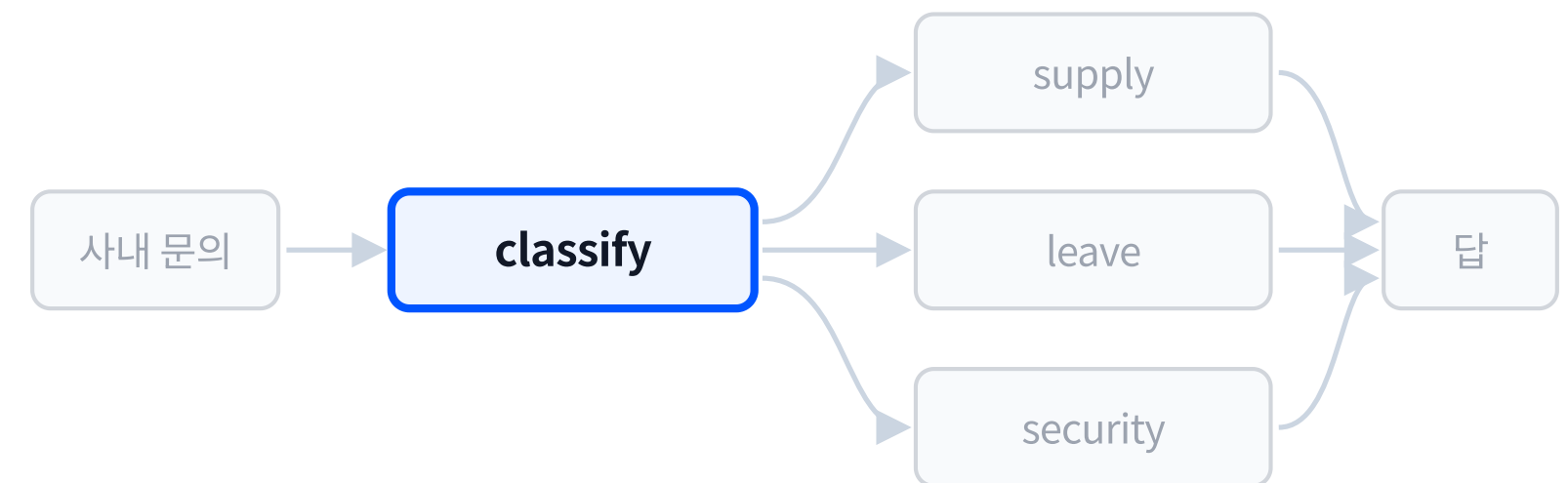
상태 정의와 분류 노드

```
class Route(BaseModel):
    step: Literal["비품", "휴가", "보안"] = Field(
        description="문의를 처리할 담당 창구")

class DeskState(TypedDict):
    question: str      # 사내 헬프데스크 문의
    decision: str       # 분류 결과
    answer: str         # 담당 창구의 답

router = llm.with_structured_output(Route)

def classify(s: DeskState) -> dict:
    d = router.invoke([
        SystemMessage(S_DESK),
        HumanMessage(s["question"]),
    ])
    return {"decision": d.step}
```



이 단계의 노드 `classify`. `Route`가 분류 결과의 스키마.

`Literal`에 적힌 세 값 밖으로는 결과가 나오지 않으므로, 판단 기준이 스키마 한 곳에 모여 있음.

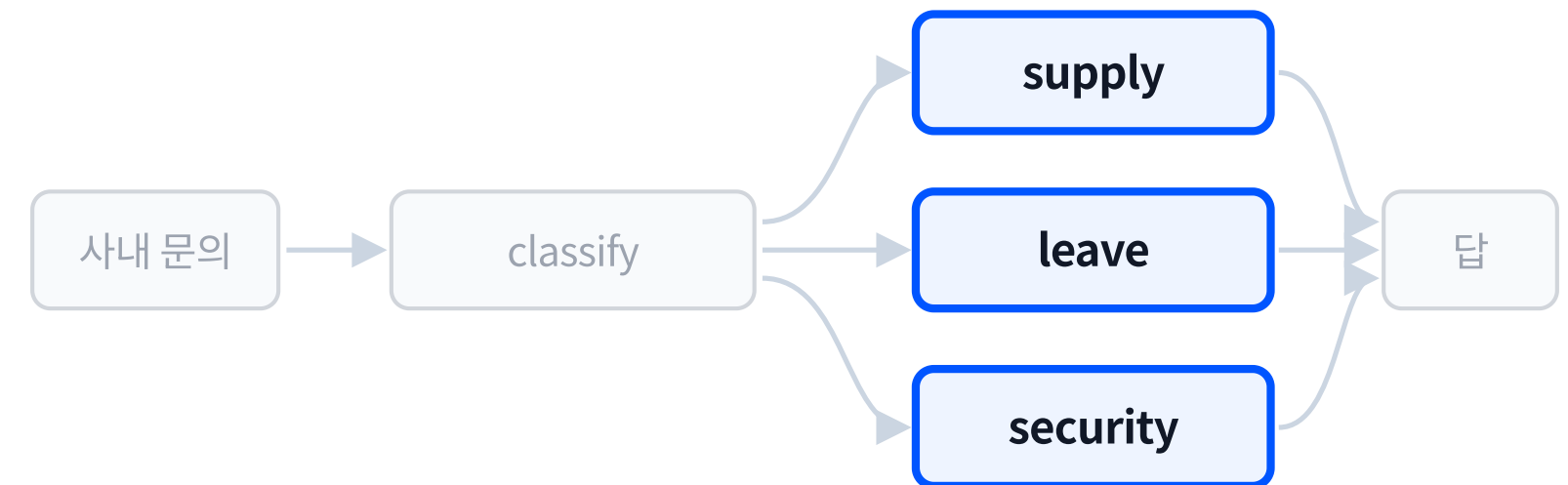
`with_structured_output(Route)`로 감싼 호출은 문자열이 아니라 `Route` 객체를 돌려줌. `classify`는 그 값을 State의 `decision` 상태 값에 넣을 뿐.

전담 노드 세 개

```
def reply_supply(s: DeskState) -> dict:
    r = llm.invoke([SystemMessage(S_SUPPLY),
                    HumanMessage(s["question"])])
    return {"answer": r.content.strip()}

def reply_leave(s: DeskState) -> dict:
    r = llm.invoke([SystemMessage(S_LEAVE),
                    HumanMessage(s["question"])])
    return {"answer": r.content.strip()}

def reply_security(s: DeskState) -> dict:
    r = llm.invoke([SystemMessage(S_SECU),
                    HumanMessage(s["question"])])
    return {"answer": r.content.strip()}
```



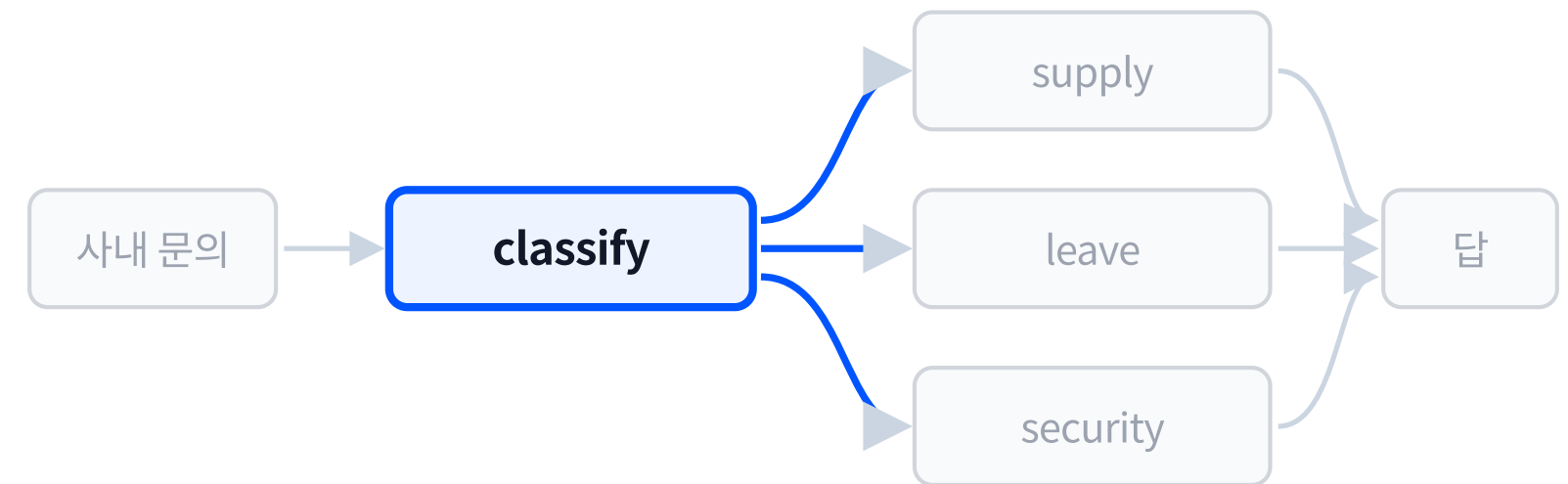
이 단계의 노드 supply · leave · security. 문의 유형 하나마다 전담 노드를 하나씩 둬. 세 함수는 같은 **DeskState**를 받아 같은 **answer** 상태 값에 쓰고, 다른 것은 창구 이름과 시스템 프롬프트뿐.

전담 노드끼리는 서로를 부르지 않음. 어느 노드가 실행될지는 이 함수들 밖의 조건부 엣지가 정함.

조건부 엣지 구성과 실행

```
# 갈 곳만 정하고 상태는 바꾸지 않는다
def route_decision(s: DeskState) -> str:
    return {"비품": "supply", "휴가": "leave",
            "보안": "security"}[s["decision"]]

g = StateGraph(DeskState)
g.add_node("classify", classify)
g.add_node("supply", reply_supply)
g.add_node("leave", reply_leave)
g.add_node("security", reply_security)
g.add_edge(START, "classify")
g.add_conditional_edges(
    "classify", route_decision,
    ["supply", "leave", "security"])
for n in ("supply", "leave", "security"):
    g.add_edge(n, END)
graph = g.compile()
out = graph.invoke({"question": Q})
print(out["decision"], "|", out["answer"])
```



이 단계에서 구성하는 것 `classify`에서 분기하는 조건부 엣지 .
실행. `add_conditional_edges`의 둘째 인자가 갈 곳을
정하는 함수, 셋째 인자가 갈 수 있는 노드 목록.
`route_decision`은 노드 이름만 돌려주고 State를 바꾸지
않음.

실행하면 `classify`가 먼저 돌아 `decision`이 정해지고, 그
값에 짝지어진 전담 노드 하나만 이어서 뚫.

실행 결과: 문의마다 다른 창구가 답합니다

```
=== 문의: 사무용 의자 높이 조절이 안 됩니다... ===  
[classify] 진입 -> decision='비품'  
[supply] 진입  
[답] 교체 신청은 비품 담당 부서의 사내  
자산관리 시스템 또는 비품 신청 창구에서  
접수해 주세요. 접수 후 처...
```

```
=== 문의: 다음 달에 사흘 연차를 붙여 쓰려는데... ===  
[classify] 진입 -> decision='휴가'  
[leave] 진입  
[답] 결재선은 부서장(팀장) 승인 후  
인사팀 확인입니다. 신청 기한은 휴가  
시작일 최소 3일 전까지입니다.
```

Multi-Agent 설계/구현

패턴 3 핸드오프(Handoff)

이동 방식 / 시스템 프롬프트 / 준비 / 구현

학습 목표

- 도구 안에서 `Command(update=...)` 로 에이전트를 가리키는 상태 변수 값을 바꿔 에이전트를 교체하는 핸드오프를 State·Node·Graph·실행 절차로 구현할 수 있습니다.
- `Command` 가 `resume` · `goto` · `update` · `graph` 네 인자를 가진 하나의 부품임을 인자별로 하는 일과 함께 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

▶ 패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

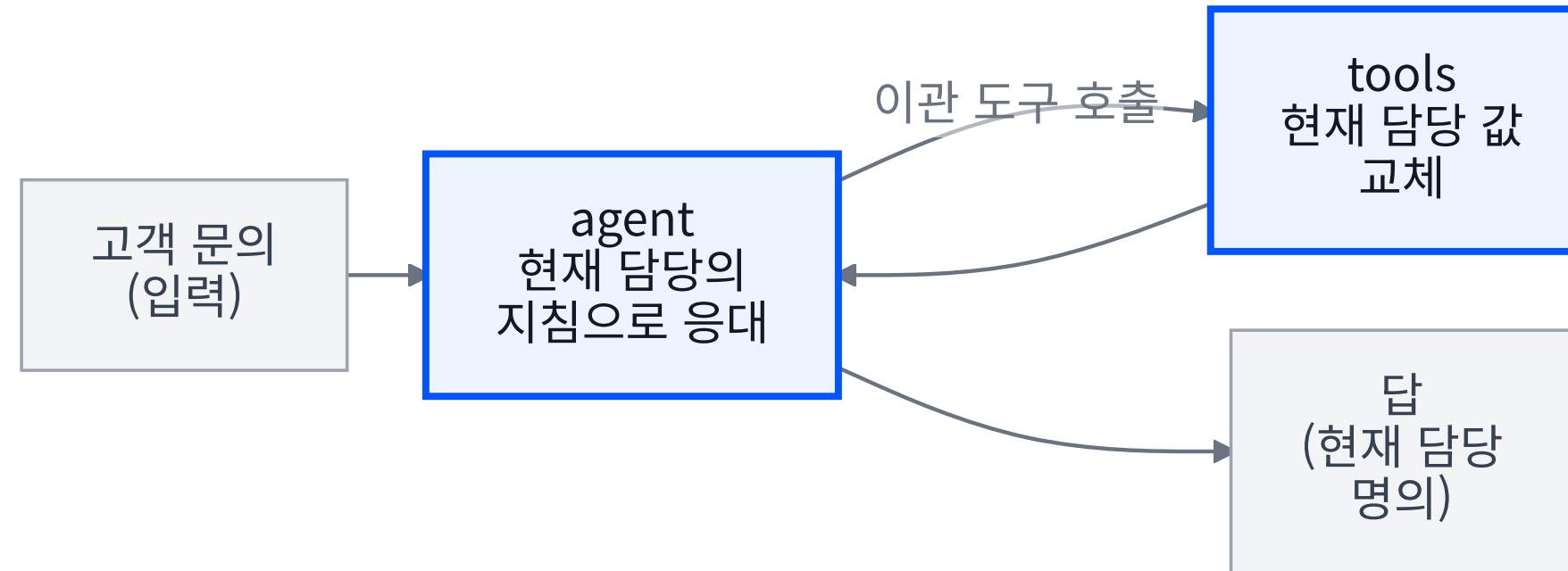
문제

고객센터 주니어 에이전트 개발자: 「주문 접수 담당에게 배송을 물어보면 「배송팀에 물어보세요」로 끝나요. 고객이 다시 전화해야 해요 ㅠㅠ」

목표

접수 담당이 받은 문의에 배송 이야기가 나오면 배송 담당에게 넘겨 그 자리에서 답이 나오게 하는 워크플로우를 만들어봅시다.

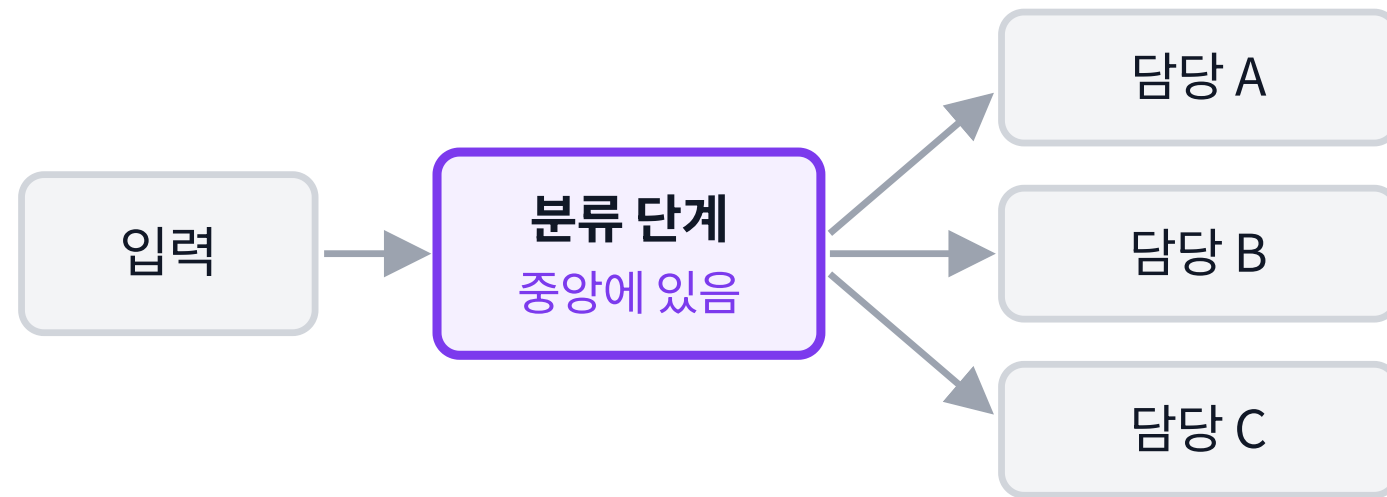
각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
agent	대화 이력과 현재 에이전트 값을 받아	현재 에이전트의 시스템 프롬프트와 도구 목록대로 응대하되, 배송 문의면 핸드오프 도구를 호출해	답 또는 도구 호출 요청을 넘긴다
tools	agent의 도구 호출 요청을 받아	핸드오프 도구를 실행해 현재 에이전트 값을 배송으로 바꿔	바뀐 에이전트 값을 상태에 넘긴다

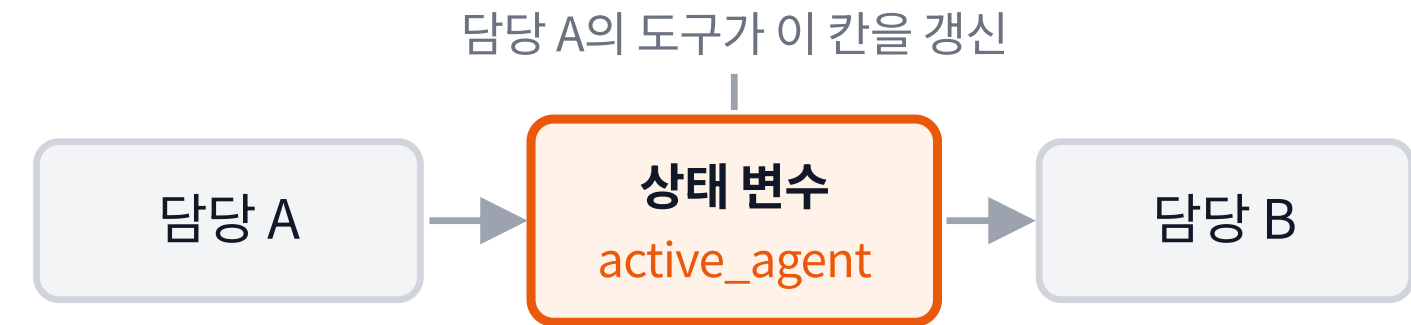
중앙이 정하는 이동과 에이전트끼리 넘기는 이동

라우팅(Router)



다음 에이전트를 정하는 곳이 **중앙의 분류 단계**.
에이전트들은 서로를 모름.

핸드오프(Handoff)



다음 에이전트를 정하는 곳이 **에이전트가 든 도구 안**.
넘기는 쪽이 다음 에이전트를 지정함.

핸드오프는 상태에 따라 행동이 달라지는 구조. 도구가 상태 변수를 갱신하면 그 값을 읽어 프롬프트와 도구가 바뀜.
상태를 갱신하면서 다음 노드까지 정해야 할 때 `Command` 를 씀.

에이전트에 따라 바뀌는 시스템 프롬프트와 도구

현재 에이전트	고정 지침(시스템 프롬프트)	도구	대화 이력
접수	주문 접수 담당이다. 주문 번호·품목만 다룬다. 배송 상태나 송장 이야기가 나오면 도구로 배송 담당에게 넘긴다.	to_delivery(배송 담당에게 넘긴다)	문의 「어제 주문한 의자가 지금 어디까지 왔나요?」
배송	배송 담당이다. 배송 상태를 두 문장으로 안내한다. 도구는 쓰지 않는다.	없음	문의 + 「배송 담당에게 넘겼습니다」

노드는 agent 하나지만, 현재 에이전트가 누구냐에 따라 시스템 프롬프트와 도구 목록이 통째로 바뀝니다. 접수 담당은 주문 번호·품목 문의면 직접 답하고, 배송 이야기가 나올 때만 넘깁니다.

상태 변수와 핸드오프 도구

```
class HandoffState(TypedDict):
    messages: Annotated[list, add_messages]
    active_agent: str # 이 칸이 행동을 바꾼다

@tool
def to_delivery(
    runtime: ToolRuntime) -> Command:
    """배송 문의는 배송 담당에게 넘긴다."""
    cid = runtime.tool_call_id
    msg = ToolMessage("배송 담당에게 넘겼습니다.",
                      tool_call_id=cid)
    return Command(update={
        "messages": [msg],
        "active_agent": "배송", # 담당 교체
    })
```



이 단계에서 만드는 것 tools 노드가 실행하는 핸드오프 도구. **active_agent**가 지금 누가 답하는지를 들고 있는 상태 변수. 도구가 돌려주는 **Command(update=)**가 이 상태 값을 바꿔 넣음.

Command는 인자에 따라 하는 일이 달라짐. **update**는 상태를 갱신하고, **goto**는 다음 노드를 지정하고, **graph**는 적용 대상 그래프를 지정하며, **resume**은 멈춰 있던 그래프를 재개함.

준비: 에이전트별 시스템 프롬프트와 문의 예시

```
ASK = "어제 주문한 의자가 지금 어디까지 왔나요?"
```

```
PROMPTS = {
```

```
    "접수": ("주문 접수 담당이다. "
```

```
        "주문 번호·품목만 다룬다. "
```

```
        "배송 상태나 송장 이야기가 나오면 "
```

```
        "도구로 배송 담당에게 넘긴다."),
```

```
    "배송": ("배송 담당이다. "
```

```
        "배송 상태를 두 문장으로 안내한다. "
```

```
        "도구는 쓰지 않는다."),
```

```
}
```

```
AGENT_TOOLS = {"접수": [to_delivery],
```

```
               "배송": []}
```

변수명	어디서 쓰나
ASK	④ 실행: 접수 창구로 들어오는 첫 문의
PROMPTS	② Node: <code>active_agent</code> 값으로 프롬프트를 고름
AGENT_TOOLS	② Node: 같은 값으로 도구 목록도 고름

이 단계에서 준비하는 것 에이전트별 시스템 프롬프트와 도구 목록. 에이전트가 바뀌면 무엇이 따라 바뀌는지가 이 두 딕셔너리에 전부 적혀 있음. 노드는 여전히 하나.

에이전트에 따라 달라지는 노드 하나

```
def agent(s: HandoffState) -> dict:
    who = s["active_agent"]
    tools = AGENT_TOOLS[who]
    m = llm
    if tools:
        m = llm.bind_tools(tools)
    sys_msg = SystemMessage(PROMPTS[who])
    res = m.invoke([sys_msg] + s["messages"])
    return {"messages": [res]}

# 도구 호출이 남았으면 tools, 아니면 끝
def need_tool(s: HandoffState) -> str:
    last = s["messages"][-1]
    return "tools" if last.tool_calls else END
```



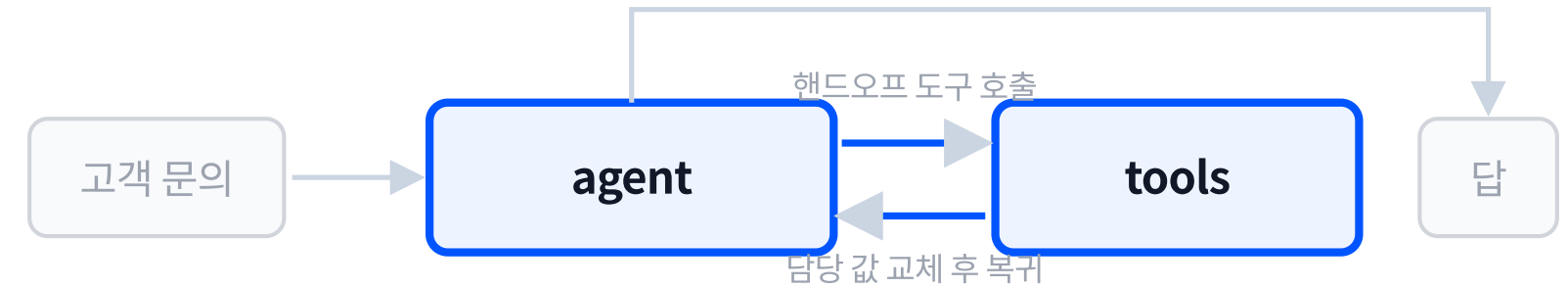
이 단계의 노드 agent. 노드는 **agent** 하나. 매 진입마다 **active_agent** 값을 읽어 **AGENT_TOOLS**에서 도구를, **PROMPTS**에서 시스템 프롬프트를 꺼냄.

에이전트마다 노드를 따로 두지 않는 구성. 에이전트가 바뀌어도 그래프 모양은 그대로이고, 바뀌는 것은 상태 변수 값.

그래프 구성과 에이전트 교체 실행

```
g = StateGraph(HandoffState)
g.add_node("agent", agent)
g.add_node("tools", ToolNode([to_delivery]))
g.add_edge(START, "agent")
g.add_conditional_edges(
    "agent", need_tool, ["tools", END])
g.add_edge("tools", "agent")
graph = g.compile()

out = graph.invoke({
    "messages": [HumanMessage(ASK)],
    "active_agent": "접수"} # 접수에서 시작
print(out["active_agent"])
print(out["messages"][-1].content)
```



이 단계에서 구성하는 것 agent ↔ tools 엣지 · 실행. 노드는 **agent**와 **tools** 둘. 도구가 갱신한 **active_agent**를 다음 **agent** 진입이 읽으므로, 에이전트를 바꾸는 전용 노드가 필요하지 않음.

실행 트레이스: agent 진입(active_agent="접수", 도구 호출 있음) → to_delivery 실행 → agent 재진입 (active_agent="배송", 도구 호출 없음). 최종 State의 active_agent = "배송".

실행 결과: 접수가 배송에게 넘기고, 배송이 답합니다

```
=== 문의: 어제 주문한 사무용 의자가 지금 어디까지 왔는지 알려주세요. ===  
[agent] 진입 active_agent='접수' tool_calls=1  
[tool] to_delivery -> active_agent 교체  
[agent] 진입 active_agent='배송' tool_calls=0  
[최종 State] active_agent='배송' 메시지 4건  
[답] 주문번호나 운송장번호가 없어 현재 배송 위치를 확인할 수 없습니다.  
해당 번호를 보내주시면 배송 상태를 안내해 드리겠습니다.
```

agent 노드가 두 번 돕니다. 첫 번째는 접수 담당의 시스템 프롬프트로(도구 호출 1건), 두 번째는 배송 담당의 시스템 프롬프트로(도구 없음). 넘긴 뒤의 에이전트는 상태에 남습니다.

Multi-Agent 설계/구현

패턴 4 슈퍼바이저(Supervisor)

문제상황과 목표 / 정의 / 시스템 프롬프트 / 준비 / 구현 / 계층형

학습 목표

- 수퍼바이저가 에이전트를 정하고 부르는 중앙 집중형 구성을 프리빌트 함수 없이 직접 만들고, 에이전트 지정과 이동을 한 노드에서 처리할 수 있습니다.
- 부모와 상태 스키마를 공유하는 서브그래프가 서브에이전트(Subagent)의 구현 형태임을 이해할 수 있습니다.
- 에이전트끼리 넘기는 핸드오프와 중앙이 정하는 수퍼바이저를 구별하고 계층형(Hierarchical)이 수퍼바이저 위에 수퍼바이저를 쌓은 구성임을 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

▶ 패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

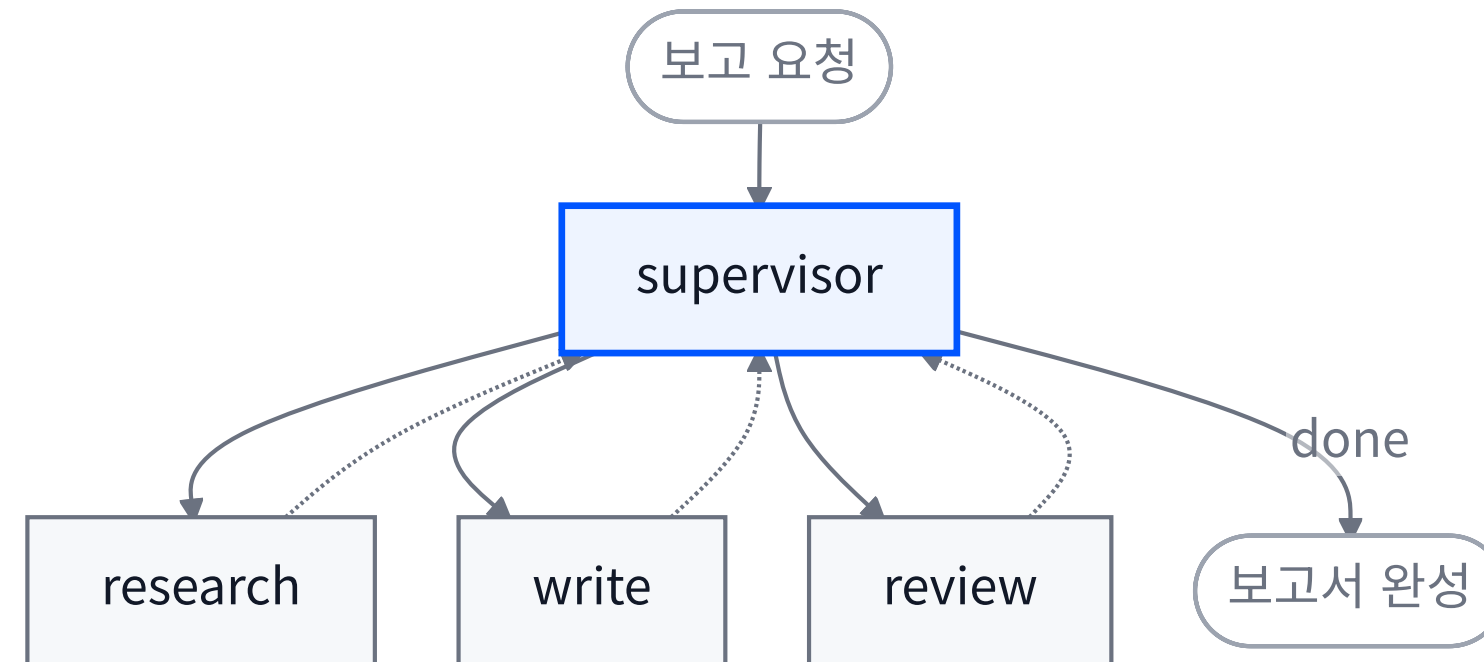
문제

총무팀 주니어 에이전트 개발자인데요, 팀장님이 "보고서 초안 좀" 하셨는데 조사→작성→검토 순서를 코드에 못 박아 놓으니 검토가 "숫자 근거 부족"으로 반려할 때마다 처음부터 다시 돌려요. 순서 하나 바꾸려면 코드를 고쳐야 하고, 반려 뒤 조사만 다시 시키는 길이 없어요 ㅠㅠ

목표

보고 요청만 주면 **팀장 역할**이 조사·작성·검토 담당을 차례로 시키고, 끝나면 **알아서 멈추는** 워크플로우를 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
supervisor	요청과 끝난 에이전트 목록을	조사·작성·검토를 한 번씩 시키고 다 끝나면 멈추는 시스템 프롬프트와 횟수 상한을 기준으로 구조화 출력(Structured Output)으로 판단해서	다음 에이전트 이름 또는 종료를 넘긴다
research	요청을	보고서에 넣을 항목을 고르는 시스템 프롬프트대로 조사해서	자료 목록을 넘긴다
write	요청과 자료 목록을	초안을 쓰는 시스템 프롬프트대로 작성해서	초안을 넘긴다
review	초안을	고칠 점 하나를 짚는 시스템 프롬프트대로 검토해서	검토 의견을 넘긴다

에이전트 순서를 코드로 못 박으면 생기는 일

확인하는 것: 조사 → 작성 → 검토를 코드의 화살표로 고정한 그래프는 검토가 반려해도 작성으로 돌아갈 수 있는가.

```
[misuse] 담당 순서를 코드 엮지로 못 박은 파이프라인 - 되돌아갈 길이 없다
[research] 진입
[write] 진입
[review] 진입 - 반려로 판정한다
```

```
노드 진입 순서: research -> write -> review -> END
```

```
최종 State: verdict='반려', reason='수치의 출처가 초안에 없다'
```

```
실행 횟수 - research 1회 / write 1회 / review 1회
```

```
[판정] 반려인데 write 재실행 0회 - 다음에 누구를 부를지 정하는 판단이 코드 엮지에만 있어 그래프가 그대로 끝났다.
```

다음 에이전트를 정하는 판단이 코드 엮지에만 있으면 실행 중에 순서를 바꿀 수 없습니다. 그 판단을 supervisor 노드 한 곳에 두는 것이 수퍼바이저 패턴입니다.

수퍼바이저(Supervisor)는 전담 에이전트(=Subagent)를 도구처럼 호출해 조율하는 중앙 에이전트입니다.

서브에이전트는 지난 대화를 기억하지 않습니다.
대화 기억은 전부 중앙 에이전트가 보관합니다.

LangChain 「Subagents」 · "a central main agent (often referred to as a supervisor) coordinates subagents by calling them as tools." (docs.langchain.com/oss/python/langchain/multi-agent/subagents, 2026-08-27 확인)

수퍼바이저와 에이전트 셋의 시스템 프롬프트

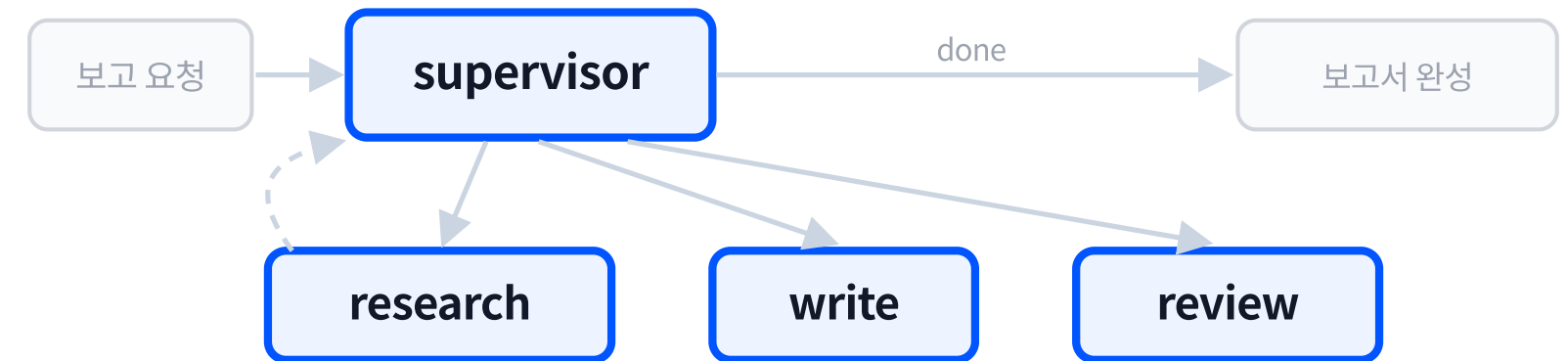
에이전트	고정 지침(시스템 프롬프트)	입력(사용자 메시지)
supervisor	research -> write -> review 순서로 한 번씩 시킨다. 다 끝났으면 done. 입력은 끝난 담당 목록이다.	요청 「회의실 예약 시스템 개편 결과 보고」 + 끝난 담당 목록
research	조사 담당이다. 보고서에 넣을 항목 2개를 한 줄로 나열한다.	요청
write	작성 담당이다. 받은 자료를 참고해 초안을 두 문장으로 쓴다.	요청 + 자료 목록
review	검토 담당이다. 초안에서 고칠 점 하나만 한 문장으로 말한다.	초안

순서는 시스템 프롬프트 한 줄이 정한 이 예시의 규칙일 뿐, 다음 에이전트는 매번 supervisor가 정합니다. 횟수 상한(4회)이 무한 반복을 막습니다.

준비: 상한과 프롬프트

```

MAX_STEPS = 4
REQ = "회의실 예약 시스템 개편 결과 보고"
S_SUP = ("research -> write -> review 순서로 "
        "한 번씩 시킨다. 다 끝났으면 done. "
        "입력은 끝난 담당 목록이다.")
S_RES = ("조사 담당이다. 보고서에 넣을 항목 "
        "2개를 한 줄로 나열한다.")
S_WRI = ("작성 담당이다. 받은 자료를 참고해 "
        "초안을 두 문장으로 쓴다.")
S_REV = ("검토 담당이다. 초안에서 고칠 점 "
        "하나만 한 문장으로 말한다.")
    
```



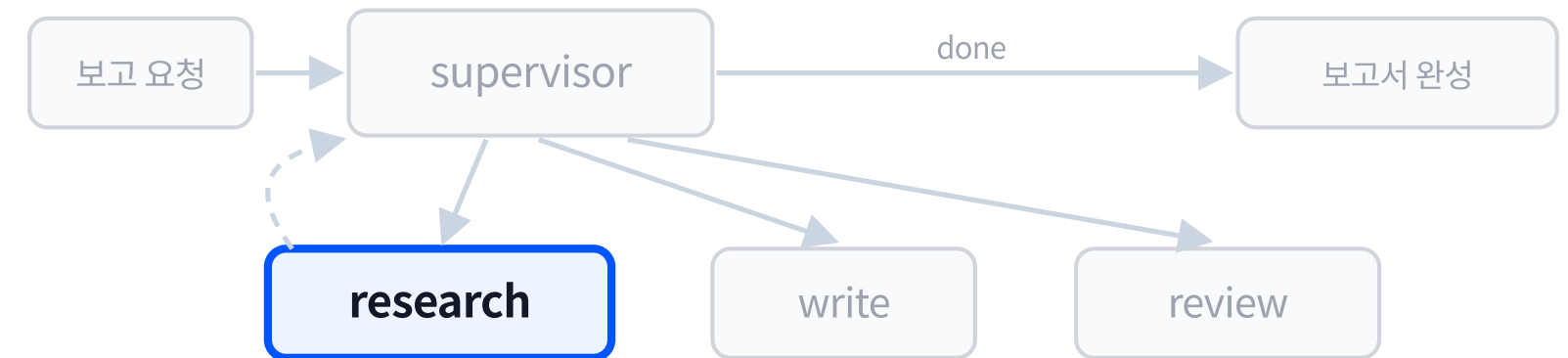
변수명	어디서 쓰나
MAX_STEPS	2단계 슈퍼바이저 Node: 진입 횟수 상한
S_SUP	2단계 슈퍼바이저 Node: 배정 기준
S_RES S_WRI S_REV	2단계 에이전트 Node 3개: 에이전트별 시스템 프롬프트
REQ	4단계 실행: 팀장이 시킨 일

supervisor와 에이전트 셋의 시스템 프롬프트를 상수로 묶어 둬. 상한이 무한 반복을 막음.

준비: research 에이전트 서브그래프

```
def collect(s: "ReportState") -> dict:
    r = llm.invoke([SystemMessage(S_RES),
                    HumanMessage(s["request"])]])
    t = "research: " + r.content.strip()[:36]
    return {"log": s["log"] + [t]}

def build_research_sub():
    g = StateGraph(ReportState) # 같은 스키마
    g.add_node("collect", collect)
    g.add_edge(START, "collect")
    g.add_edge("collect", END)
    return g.compile()
```

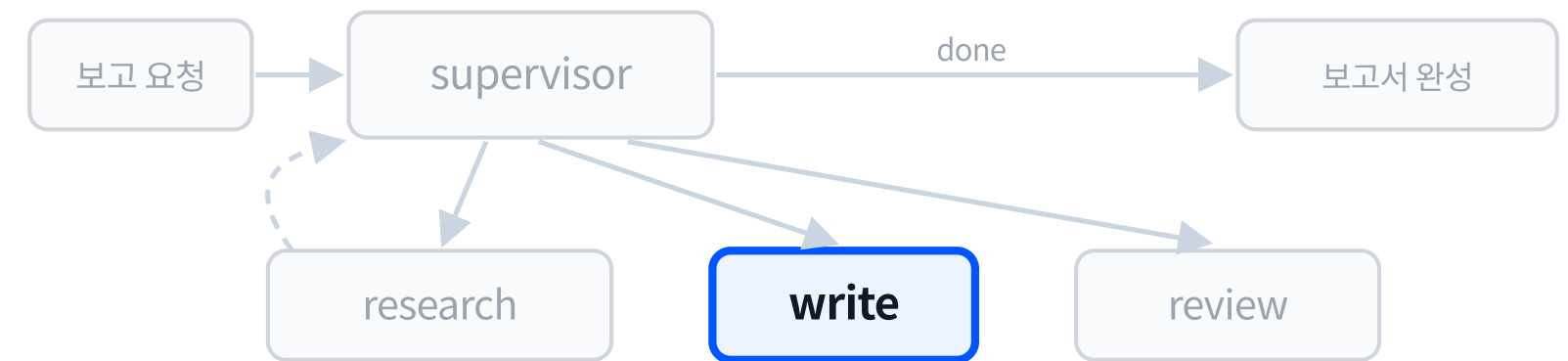


코드 항목	어디서 쓰나
collect	서브그래프 안의 유일한 Node
build_research_sub	3단계 Graph: add_node("research", ...)
"research: ..."	log 항목 형식: 에이전트 이름 + 콜론

research 노드의 코드. log 값 전체를 state["log"] + [새 줄] 로 통째 돌려줌. 부모 리듀서와 겹쳐 두 번 쌓이지 않게 리듀서를 쓰지 않음.

준비: write 에이전트 Node

```
def write_draft(s: "ReportState") -> dict:
    h = s["request"] + "\n자료: " + str(s["log"])
    r = llm.invoke([SystemMessage(S_WRI),
                    HumanMessage(h)])
    log = s["log"] + ["write: 초안 작성"]
    return {"draft": r.content.strip(),
            "log": log}
```



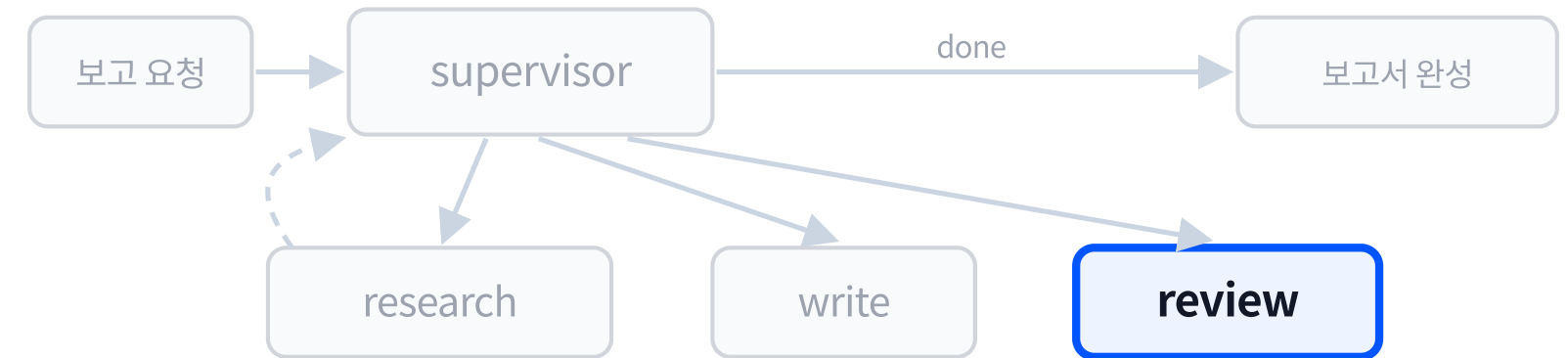
코드 항목	어디서 쓰나
write_draft	3단계 Graph: build_write_sub() 안의 Node
s["log"]	앞 에이전트가 남긴 기록을 초안 재료로 읽음
"write: ..."	같은 log 항목 형식

write 노드의 코드. 앞 에이전트의 결과를 상태에서 읽어 초안을 쓰므로, 순서를 슈퍼바이저가 정하는 의미가 생김.

준비: review 에이전트 서브그래프

```
def review(s: "ReportState") -> dict:
    r = llm.invoke([SystemMessage(S_REV),
                    HumanMessage(s["draft"])]])
    t = "review: " + r.content.strip()[:36]
    return {"log": s["log"] + [t]}

def build_review_sub():
    g = StateGraph(ReportState)
    g.add_node("check", review)
    g.add_edge(START, "check")
    g.add_edge("check", END)
    return g.compile()
```



코드 항목	어디서 쓰나
review	서브그래프 안의 유일한 Node
build_review_sub	3단계 Graph: <code>add_node("review", ...)</code>
노드 이름 "check"	부모의 노드 이름과 달라도 무방함

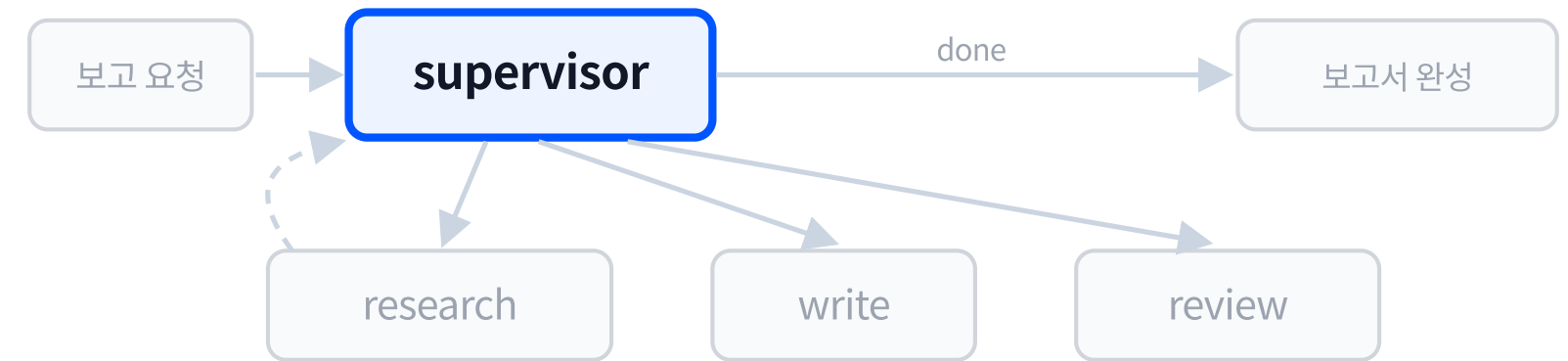
review 노드의 코드. 서브그래프 세 개가 모두 부모와 같은 스키마를 씀. 그래서 별도 변환 없이 노드로 엮힘.

1단계: 상태와 에이전트 배정 스키마

```
class ReportState(TypedDict):
    request: str
    log: list      # 리듀서 금지: 중복 누적 방지
    draft: str
    step: int      # 상한 판정용

class Assign(BaseModel):
    who: Literal["research", "write",
                "review", "done"]

assigner = llm.with_structured_output(Assign)
```



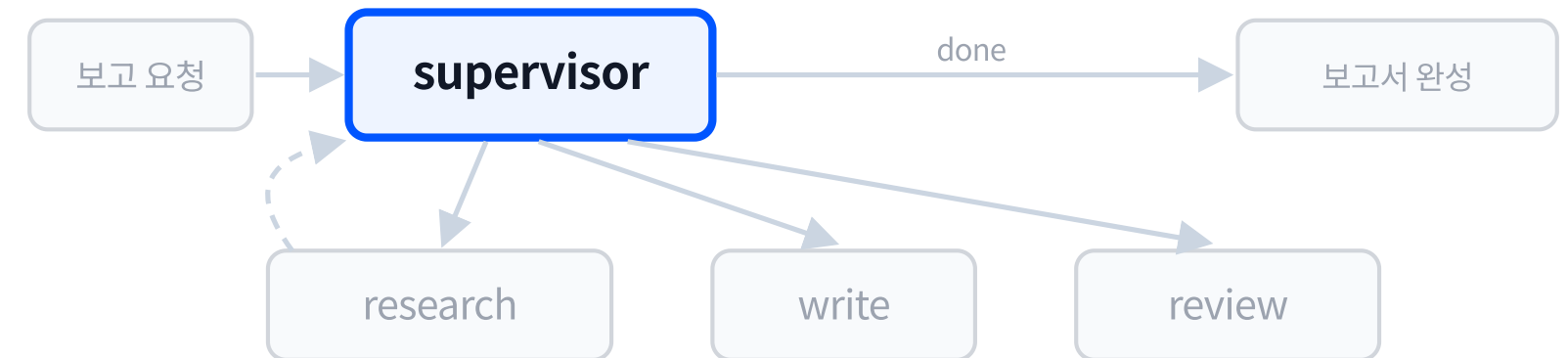
supervisor 노드의 판정 형식. **step**은 중앙 에이전트에 들어온 횟수. 이 상태 값 하나가 무한 반복을 막는 상한의 근거가 됨.

Assign은 다음 차례로 일할 에이전트의 이름만 담는 스키마. 에이전트 후보와 종료 신호(**done**)가 같은 목록에 들어감.

with_structured_output을 통과한 응답은 who에 적힌 값 (research·write·review·done) 밖으로 나오지 않음.

2단계: 에이전트를 정하면서 이동까지 하는 노드

```
# 상태도 바꾸고 갈 곳도 정한다 -> 그래서 Command
def supervisor(s: ReportState) -> Command:
    if s["step"] >= MAX_STEPS:
        return Command(goto=END)
    done = [x.split(":")[0] for x in s["log"]]
    a = assigner.invoke([
        SystemMessage(S_SUP),
        HumanMessage(REQ + str(done))]
    )
    nxt = END if a.who == "done" else a.who
    up = {"step": s["step"] + 1}
    return Command(goto=nxt, update=up)
```



supervisor 노드. 상태를 바꾸면서 갈 곳도 정해야 할 때 반환형이 **Command**가 됨. 조건부 엣지는 갈 곳만 정하고 상태를 바꾸지 않음.

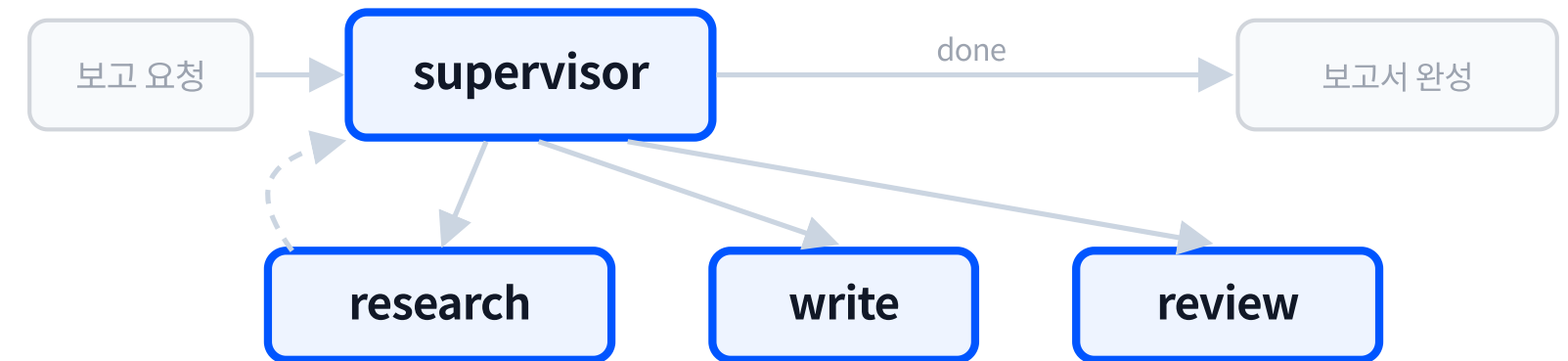
goto가 이동, **update**가 상태 갱신. 에이전트 이름이 그대로 다음 노드 이름이 됨.

끝난 에이전트 목록을 프롬프트에 실어 보냄. 서브에이전트는 지난 대화를 기억하지 않으므로 필요한 맥락은 슈퍼바이저가 명시해서 실어 보냄.

3~4단계: 서브에이전트 등록과 슈퍼바이저 복귀

```
def build_write_sub():
    g = StateGraph(ReportState) # 부모와 같은 스키마
    g.add_node("draft", write_draft)
    g.add_edge(START, "draft")
    g.add_edge("draft", END)
    return g.compile()

g = StateGraph(ReportState)
g.add_node("supervisor", supervisor)
g.add_node("research", build_research_sub())
g.add_node("write", build_write_sub())
g.add_node("review", build_review_sub())
g.add_edge(START, "supervisor")
for n in ("research", "write", "review"):
    g.add_edge(n, "supervisor") # 끝나면 중앙으로
graph = g.compile()
out = graph.invoke({"request": REQ, "log": [],
                    "draft": "", "step": 0})
```



에이전트 셋이 슈퍼바이저로 돌아오는 엣지와 실행 부분.
컴파일된 서브그래프가 그대로 노드 위치에 들어감. 부모와 같은 스키마를 쓰므로 별도 변환 없이 엮힘. **이 구조에서 서브에이전트는 서브그래프로 구현됨.**

에이전트 셋은 모두 **supervisor**로 돌아옴. 다음 에이전트를 정하는 판단이 supervisor 노드 한 곳에만 있어 중앙 집중형이 성립함.

공유 스키마 서브그래프에 누적 리듀서를 함께 걸면 같은 항목이 두 번 쌓임. 서브그래프가 누적된 값 전체를 돌려주고 부모 리듀서가 그것을 다시 이어붙이기 때문.

실행 결과: 슈퍼바이저가 에이전트 셋을 차례로 부르고 멈춥니다

```
=== 요청='회의실 예약 시스템 개편 결과 보고' ===  
[supervisor] 진입 step=0 -> who='research'  
    [sub:research/collect] 자료 항목 뽑기  
[supervisor] 진입 step=1 -> who='write'  
    [sub:write] 초안 쓰기  
[supervisor] 진입 step=2 -> who='review'  
    [sub:review] 검수  
[supervisor] 진입 step=3 -> who='done'  
[최종 State] step=4 log 3건  
    - research: 개편 주요 내용, 운영 성과 및 개선 효과  
    - write: 초안 작성  
    - review: '항상'과 '개선'이 반복되어 의미가 중복되므로...
```

에이전트가 끝날 때마다 슈퍼바이저로 돌아오고, 슈퍼바이저가 끝난 에이전트 목록을 보고 다음을 정합니다. 네 번째 진입에서 done을 골라 스스로 멈춥니다.

[참고] 계층형(Hierarchical): 수퍼바이저를 관리하는 수퍼바이저

전담 에이전트 위치에 또 하나의 수퍼바이저가 있으면 같은 구조가 층으로 쌓임.



파란 화살표는 배정, 회색 점선은 결과 복귀 · 층이 늘어도 두 화살표의 짝은 같음

langgraph-supervisor-py README · "Hierarchical systems are a type of multi-agent architecture where specialized agents are coordinated by a central supervisor agent." · "You can create multi-level hierarchical systems by creating a supervisor that manages multiple supervisors." (github.com/langchain-ai/langgraph-supervisor-py, 2026-08-27 확인)

Multi-Agent 설계/구현

패턴 5 오케스트레이터-워커 (Orchestrator-Workers)

문제상황과 목표 / 동적 분해 / 시스템 프롬프트 / 준비 / 구현

학습 목표

- 하위 작업의 개수와 내용을 입력을 보고 실행 시점에 정하는 오케스트레이터-워커 그래프를 구성할 수 있습니다.
- `Send` 로 워커를 띄우고 리듀서로 결과를 합치는 그래프를 구성할 수 있습니다.
- 오케스트레이터-워커와 병렬화를 구별하여 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 슈퍼바이저

▶ 패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

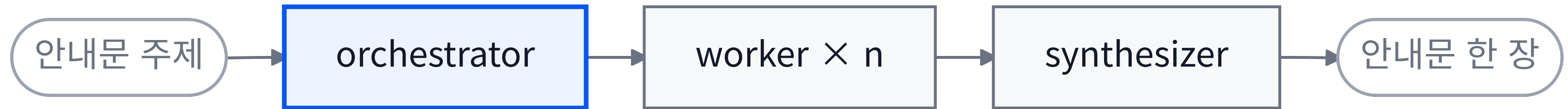
문제

인사팀 주니어 에이전트 개발자인데요, 안내문은 주제마다 문단 수가 달라요. 두 문단짜리도 네 문단짜리도 있어서 틀을 미리 못 정해요 ㅠㅠ

목표

인사팀이 안내문 주제만 주면, **필요한 문단을 먼저 정하고** 문단마다 작성 담당을 붙여 쓴 뒤 **한 장으로 모아** 주게 만들어봅시다.

각 단계별 요구사항



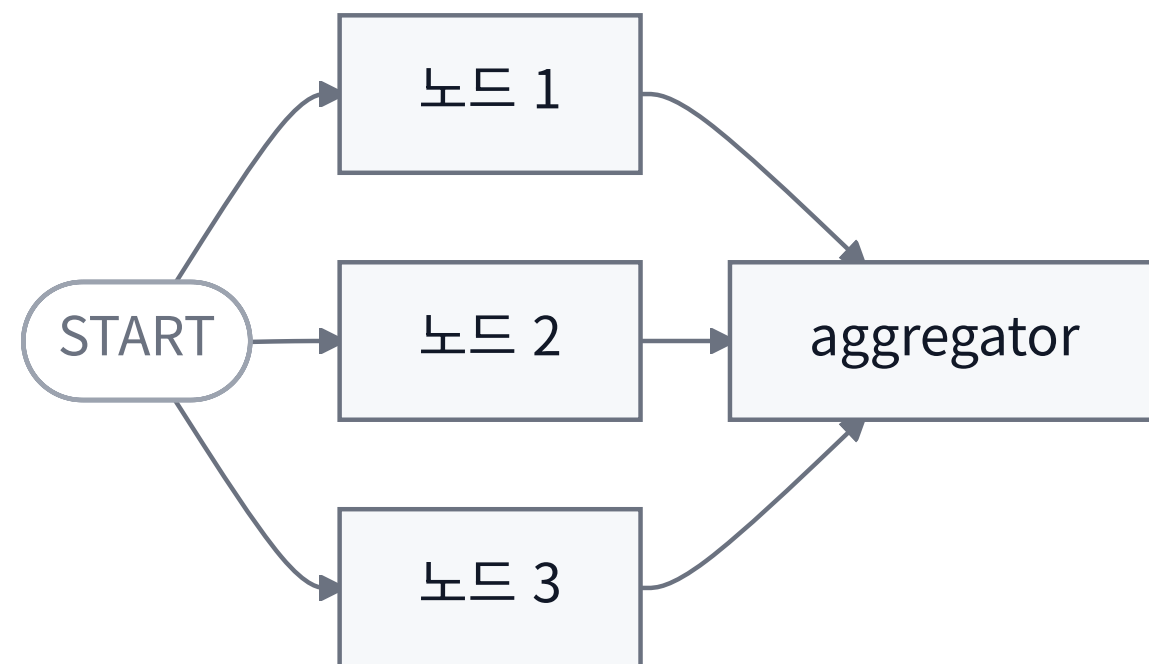
노드	무엇을 받아	어떻게	무엇을 넘긴다
orchestrator	안내문 주제를	문단 계획을 세우는 시스템 프롬프트대로 구조화 출력 (Structured Output)으로 계획해서	문단 계획 목록을 넘기고, 계획 개수만큼 worker를 띄운다
worker × n	문단 하나의 제목과 내용 한 줄을	두 문장으로 쓰는 시스템 프롬프트대로 작성해서	쓴 문단을 넘긴다
synthesizer	쓴 문단 전부를	순서대로 이어 붙여서	안내문 한 장을 넘긴다

하위 작업 개수를 정하는 시점

설계 시점에 정함

"팬아웃될 노드 수가 코드에 있다"

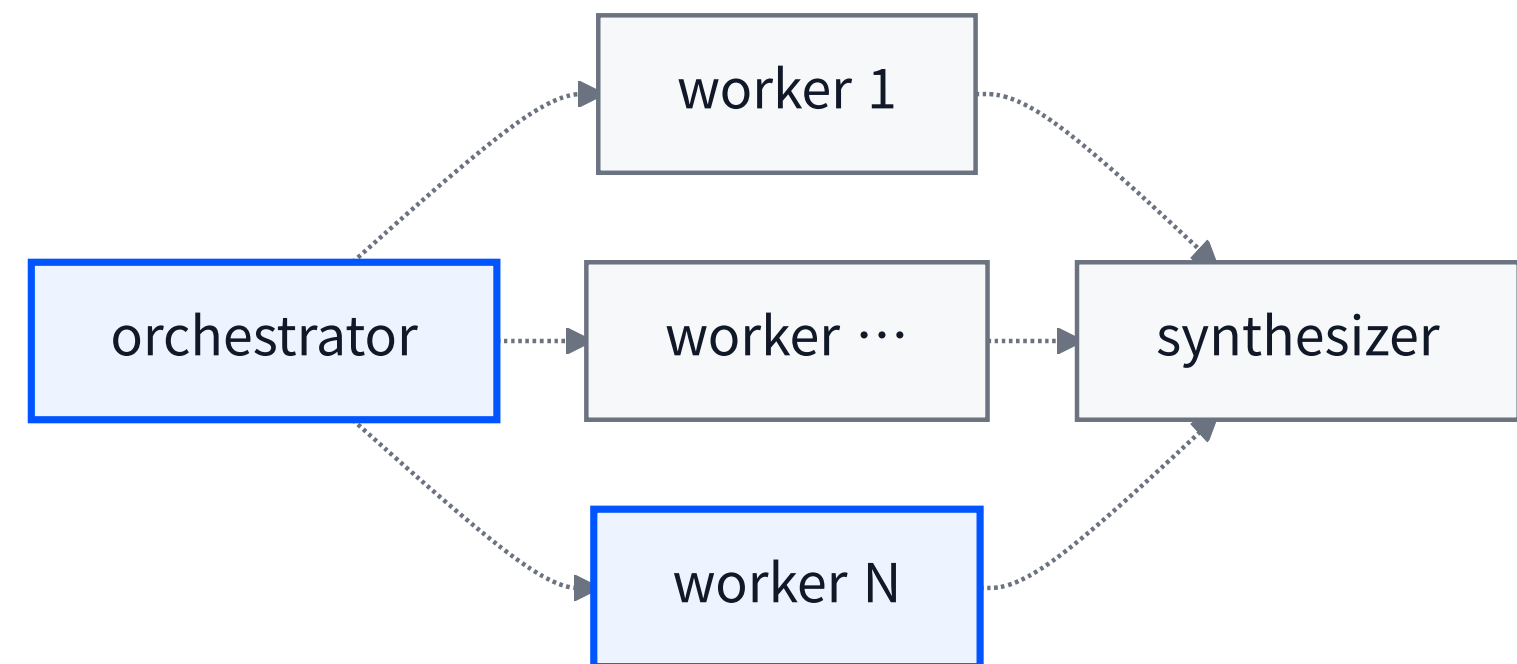
그래프를 그리는 단계에서 하위 작업 수가 이미 확정됨. 입력이 무엇이든 팬아웃되는 노드 수는 같음.



실행 시점에 정함

"팬아웃될 노드 수가 입력에 있다"

오케스트레이터가 입력을 읽어 하위 작업을 나누고, 나온 개수만큼 워커를 팬아웃함. 개수를 코드에 못 박을 수 없는 입력에 오케스트레이터-워커를 씀.



오케스트레이터가 하는 일

하위 작업은 미리 정해지지 않고, 오케스트레이터가 입력을 보고 정함.

"...subtasks aren't pre-defined, but determined by the orchestrator based on the specific input."

- 과제를 하위 작업으로 나눈다
- 하위 작업을 워커에 위임한다
- 워커의 출력을 종합한다

오케스트레이터와 워커의 시스템 프롬프트

에이전트	고정 지침(시스템 프롬프트)	입력(사용자 메시지)
orchestrator	사내 안내문 문단 계획을 필요한 만큼(2~4개) 세운다.	주제 「사내 메신저 보안 수칙 안내」
worker × n	받은 문단을 두 문장으로 쓴다.	문단 제목 + 이 문단에 답을 내용 한 줄
synthesizer	시스템 프롬프트 없음: 쓴 문단을 순서대로 잇는 순수 함수	사용자 메시지 없음: 상태의 쓴 문단 전부를 읽음

orchestrator의 답은 「문단 제목 · 내용 한 줄」 목록으로 고정된 형식(구조화 출력)입니다. 목록의 길이가 곧 worker의 수입니다.

준비: 시스템 프롬프트와 입력 예시

```
D1 = "문단 제목"
D2 = "이 문단에 담을 내용 한 줄"
S_PLAN = ("사내 안내문 문단 계획을 "
          "필요한 만큼(2~4개) 세운다.")
S_SEC = "받은 문단을 두 문장으로 쓴다."
GUIDE = "사내 메신저 보안 수칙 안내"
```



변수명	어디서 쓰나
D1 D2	1단계 State: Section 필드의 description
S_PLAN	2단계 오케스트레이터 Node: 계획 프롬프트
S_SEC	2단계 워커 Node: 문단 집필 프롬프트
GUIDE	4단계 실행: 안내문 주제

시스템 프롬프트와 주제를 상수로 묶어 둠.

D1 · D2 는 모델이 읽는 필드 설명이므로, 스키마 자체가 프롬프트 역할을 함.

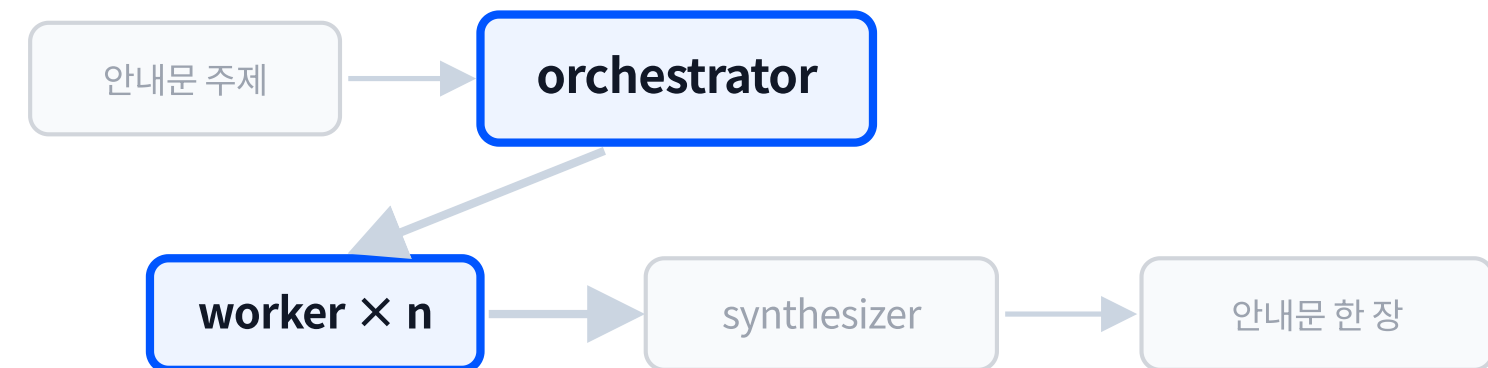
상태 정의: 리듀서와 워커 전용 상태

```
class Section(BaseModel):  
    name: str = Field(description=D1)  
    description: str = Field(description=D2)
```

```
class Sections(BaseModel):  
    sections: list[Section]
```

```
class GuideState(TypedDict):  
    topic: str  
    sections: list  
    # 워커들이 동시에 쓴다 -> 리듀서 필수  
    written: Annotated[list, operator.add]  
    guide: str
```

```
class WorkerState(TypedDict): # 워커 하나의 몫  
    section: Section  
    written: Annotated[list, operator.add]
```



orchestrator의 계획 형식과 worker의 몫을 담는 상태.

Annotated[list, operator.add]가 리듀서(reducer).

여러 워커가 같은 키에 동시에 쓸 때 값을 덮어쓰지 않고
이어붙이도록 함. 리듀서가 없으면 한 스텝에 한 값만 받는다는
오류로 멈춤.

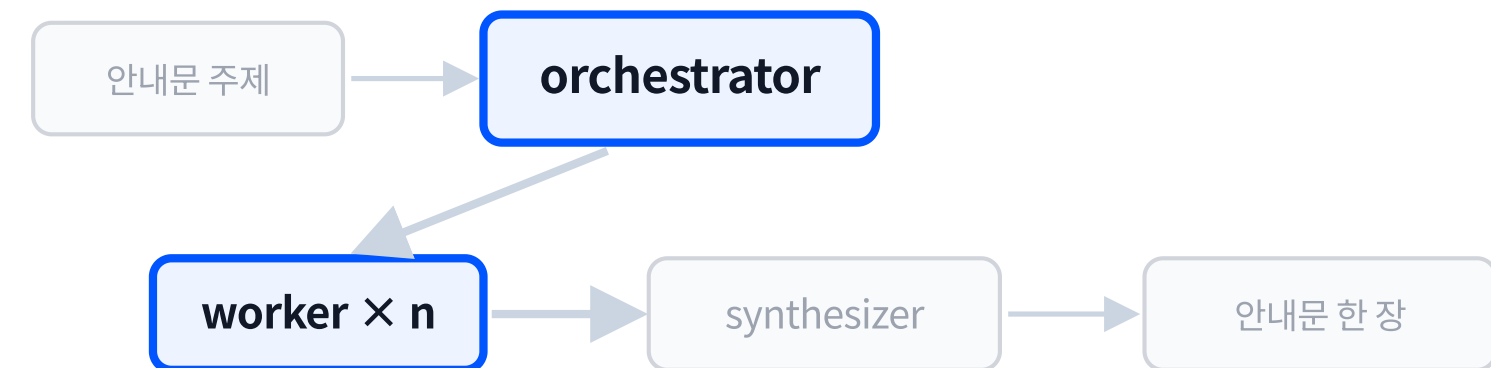
WorkerState는 워커 하나가 받는 몫만 담은 별도 상태. 워커는
전체 상태가 아니라 자기 문단 하나와 결과 상태 값만 봄.

노드 준비: 오케스트레이터와 워커

```
planner = llm.with_structured_output(Sections)
```

```
def orchestrator(s: GuideState) -> dict:
    plan = planner.invoke([
        SystemMessage(S_PLAN),
        HumanMessage(s["topic"])]])
    return {"sections": plan.sections}
```

```
def worker(s: WorkerState) -> dict:
    sec = s["section"]
    h = sec.name + "\n" + sec.description
    r = llm.invoke([SystemMessage(S_SEC),
                    HumanMessage(h)])
    return {"written": [r.content]}
```



orchestrator 노드와 worker 노드.

with_structured_output(Sections)로 계획을

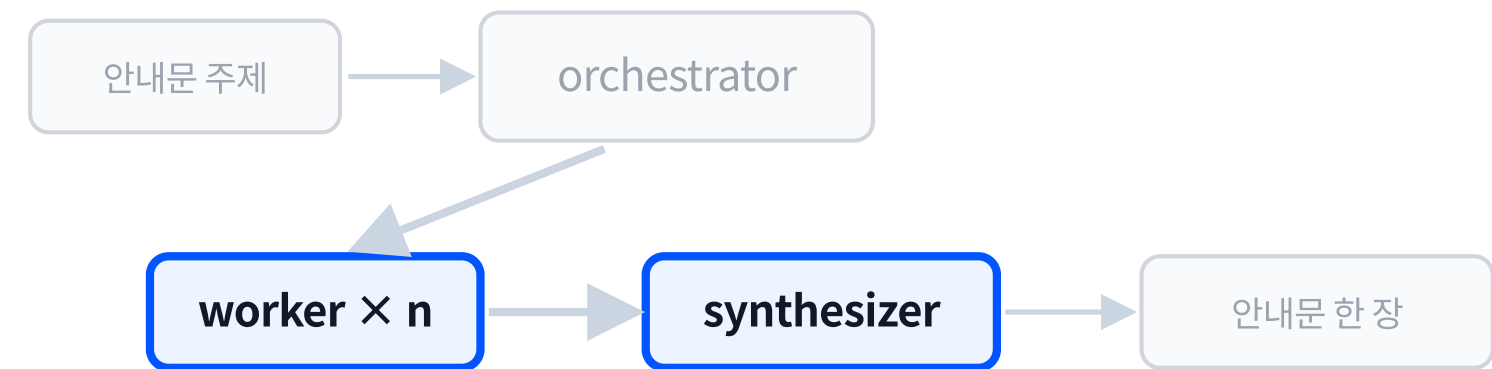
pydantic 객체로 받음. 오케스트레이터의 출력은 자유 텍스트가 아니라 문단 목록이라는 스키마이며, 목록의 길이가 곧 워커 수가 됨.

worker는 자기 몫 하나만 처리하고 결과를 리스트 항목 하나로 돌려줌. 돌려준 상태 값은 리듀서가 이어붙여 줌.

합치기 노드와 Send 목록

```
def synthesizer(s: GuideState) -> dict:
    joined = "\n\n".join(s["written"])
    return {"guide": joined}

# 노드 이름 대신 Send 목록을 돌려준다
# - 워커 '개수'가 여기서 정해진다
def assign_workers(s: GuideState):
    return [Send("worker", {"section": x})
            for x in s["sections"]]
```



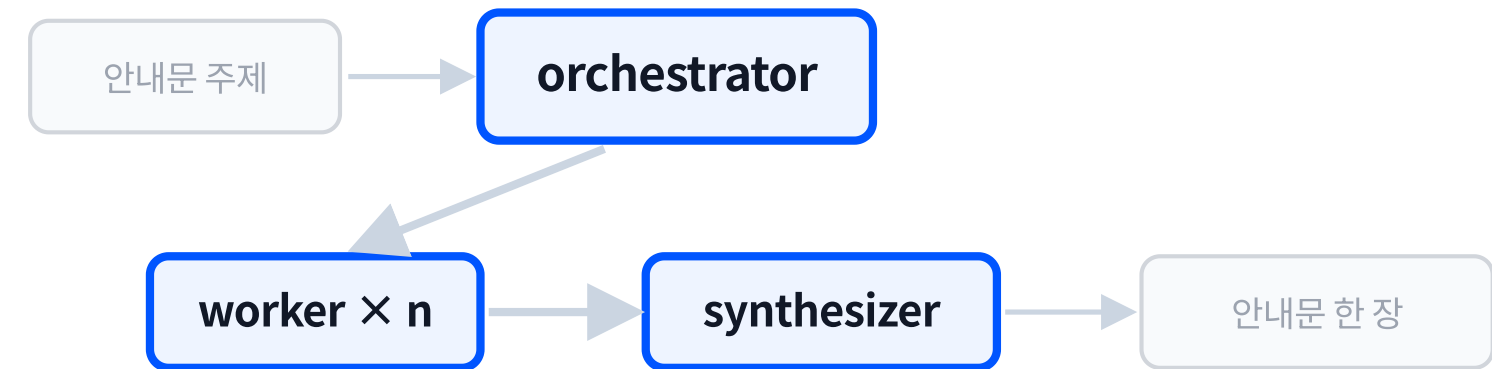
synthesizer 노드와 worker를 팬아웃하는 화살표. **Send** 한 쌍 = 워커 이름 + 그 워커가 받을 상태. 오케스트레이터-워커에서 워커를 팬아웃하는 수단이 이 **Send** 목록이 됨.

assign_workers가 돌려주는 목록의 길이가 이번 실행에서 팬아웃될 워커 수. 계획이 길면 워커도 그만큼 늘어남.

그래프 구성과 실행

```
g = StateGraph(GuideState)
g.add_node("orchestrator", orchestrator)
g.add_node("worker", worker)
g.add_node("synthesizer", synthesizer)
g.add_edge(START, "orchestrator")
g.add_conditional_edges(
    "orchestrator", assign_workers,
    ["worker"])
g.add_edge("worker", "synthesizer")
g.add_edge("synthesizer", END)
graph = g.compile()

IN = {"topic": GUIDE, "sections": [],
      "written": [], "guide": ""}
out = graph.invoke(IN)
print(len(out["written"]), "개 워커가 펼쳐졌다")
```



그래프 연결과 실행 부분. `add_conditional_edges`의 판단 함수가 노드 이름 대신 `Send` 목록을 돌려줌. 세 번째 인자는 이 분기가 닿을 수 있는 노드를 그래프에 알려 주는 목록.

실행 트레이스: `orchestrator` 진입으로 문단 계획이 서고, 계획된 문단마다 `worker`가 진입한 뒤, `synthesizer`가 한 번 진입해 부분 결과를 이어붙여 줌. 최종 State의 `written`에 워커 결과가 모두 담김.

실행 결과: 계획이 4개라서 worker 노드도 4개 팬아웃됩니다

```
=== 주제='사내 메신저 보안 수칙 안내' ===  
[orchestrator] 진입 -> 문단 4개 계획  
  [worker] 진입: 사내 메신저 보안의 중요성  
  [worker] 진입: 메시지 및 파일 공유 수칙  
  [worker] 진입: 계정 및 기기 보안  
  [worker] 진입: 보안 사고 발생 시 대응  
[synthesizer] 진입 -> 조각 4개 합침  
[최종 State] 펼쳐진 워커 4개
```

```
--- 안내문 ---  
## 사내 메신저 보안의 중요성  
사내 메신저로 공유되는 업무 정보와 개인정보는  
유출되지 않도록 각별히 보호해야 합니다. 안전한  
메신저 사용을 위해 비밀번호 관리, 의심스러운 링크  
주의 등 기본 보안 수칙을 반드시 준수해야 합니다.  
  
## 메시지 및 파일 공유 수칙  
메시지와 파일을 공유할 때는 기밀정보와 개인정보의  
전송을 최소화하고, 수신자를 확인한 뒤 적절한 파일  
권한을 설정해야 합니다. 출처가 불분명한 링크와  
첨부파일은 열기 전에 주의 깊게 확인해야 합니다.
```

```
## 계정 및 기기 보안  
강력한 비밀번호와 다중 인증을 사용해 계정을  
보호해야 합니다. 자리를 비울 때는 화면을 잠그고,  
공용·개인 기기 사용 시 보안 설정을 준수해야  
합니다.  
  
## 보안 사고 발생 시 대응  
보안 사고가 발생하면 오발송을 즉시 취소하고  
비밀번호를 변경해야 합니다. 의심스러운 메시지를  
받았거나 계정 탈취가 의심될 경우에는 지체 없이  
보안 담당 부서에 신고합니다.
```

Multi-Agent 설계/구현

패턴 6 평가자-최적화자(Evaluator-Optimizer)

문제상황과 목표 / 종료 조건 / 시스템 프롬프트 / 준비 / 되돌림 루프

학습 목표

- 만드는 일과 판정하는 일을 평가자-최적화자(Evaluator-Optimizer) 패턴으로 나누어 설계할 수 있습니다.
- 평가자 위치에 사람 승인을 앓힌 구성과 LLM 평가자를 앓힌 구성을 나란히 놓고, 두 구성이 무엇을 공유하고 무엇이 다른지 설명할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

▶ 패턴 6 평가자-최적화자

실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

문제

총무팀 주니어 에이전트 개발자인데요, 회의록 요약에 에이전트 하나에 맡기고
믿었는데 결정 사항은 빠지고 담당자는 틀리고 형식도 제멋대로... 그대로 팀장님께
보냈다가 혼났어요 ㅠㅠ

목표

요약을 만들면 다른 쪽이 결정 사항과 담당자가 다 있는지 **검사**하고, 빠졌으면 이유를
붙여 **다시 쓰게 하는** 워크플로우를 만들어봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
generator	회의록과 반려 사유를	짧게 요약하되 반려 사유가 있으면 반영하는 시스템 프롬프트대로 작성해서	요약과 시도 횟수를 넘긴다
evaluator	회의록과 요약을	결정된 것과 담당자가 다 있는지를 기준으로 구조화 출력 (Structured Output)으로 판단해서	합격이거나 시도 횟수가 상한이면 끝내고, 반려이면 고칠 점 한 문장을 붙여 generator로 되돌린다

종료 조건은 **코드가 참·거짓으로 판정할 수 있는 것**이어야 합니다.

「충분히 좋아지면」은 그런 조건이 아닙니다.

평가자-최적화자(Evaluator-Optimizer)는 한 호출이 응답을 만들고 다른 호출이 그 응답을 평가하는 구성입니다. 생성 노드와 평가 노드를 한 그래프에 두고 반려 사유를 실어 되돌리는 루프가 Critic 에이전트 자기 개선 루프입니다. 평가 기준이 뚜렷하고 되풀이해 다듬을수록 좋아지는 일에 맞습니다.

정의: LangChain 「Workflows and agents」 · "In evaluator-optimizer workflows, one LLM call creates a response and the other evaluates that response."

(docs.langchain.com/oss/python/langgraph/workflows-agents, 2026-08-27 확인) · 용도: Anthropic 「Building effective agents」 · "This workflow is particularly effective when we have clear evaluation criteria, and when iterative refinement provides measurable value." (anthropic.com/engineering/building-effective-agents, 2026-08-27 확인)

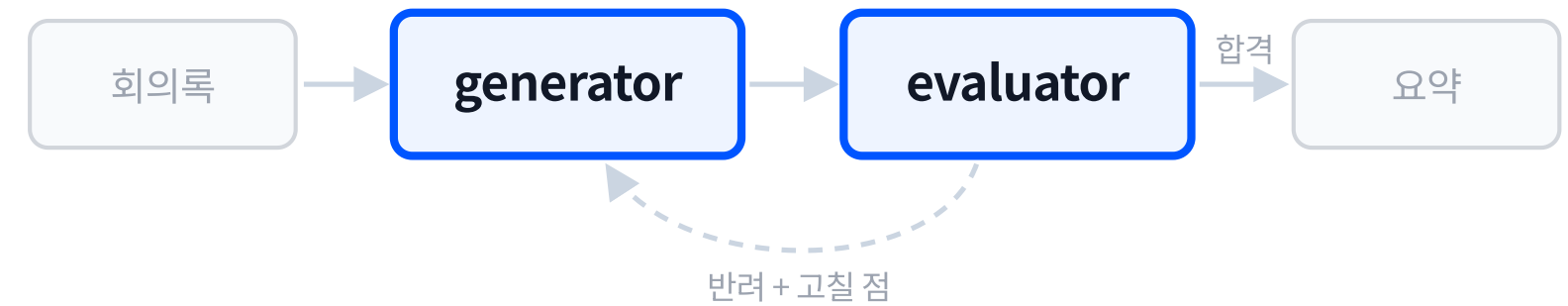
생성자와 평가자의 시스템 프롬프트

에이전트	고정 지침(시스템 프롬프트)	입력(사용자 메시지)
generator (첫 시도)	회의록을 25자 이내로 아주 짧게 요약한다.	회의록 본문
generator (반려 뒤)	회의록을 한 문장으로 요약한다. 지난 지적을 반영한다.	지적(고칠 점) + 회의록 본문
evaluator	요약에 '결정된 것'과 '담당자'가 다 있으면 합격, 아니면 반려.	회의록 본문(근거 자료) + 요약

준비: 시스템 프롬프트와 입력 예시

```
MAX_TRIES = 3
D3 = "반려라면 무엇을 고칠지 한 문장"
S_GEN1 = ("회의록을 25자 이내로 "
          "아주 짧게 요약한다.")
S_OPT = ("회의록을 한 문장으로 요약한다. "
          "지난 지적을 반영한다.")
S_EVAL = ("요약에 '결정된 것'과 '담당자'가 "
          "다 있으면 합격, 아니면 반려.")
```

시스템 프롬프트와 회의록을 상수로 묶어 둬. `S_*` 는 노드의 시스템 프롬프트, `P6_MINUTES` 와 `IN6` 은 실행 입력.



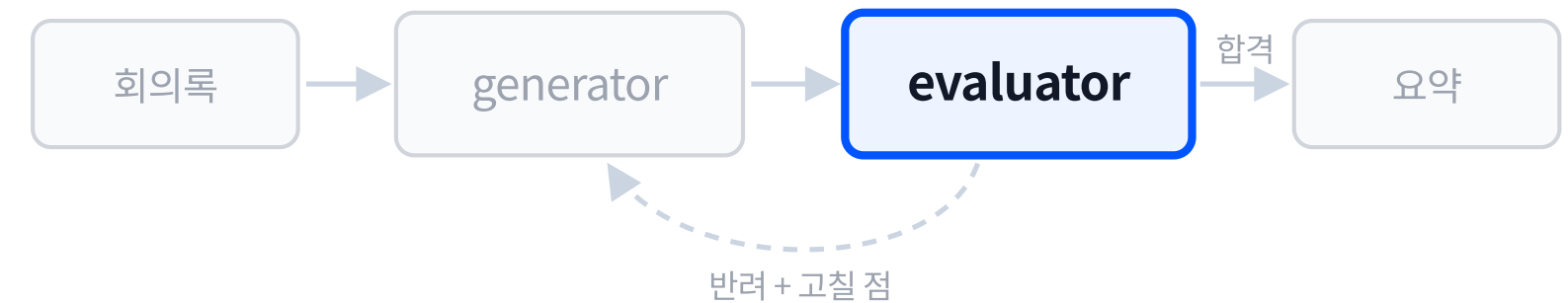
```
P6_MINUTES = (
    "회의실 예약 시스템을 다음 달부터 "
    "사내 포털로 옮기기로 했다. "
    "이전 작업은 총무팀이 맡고, "
    "사용법 안내는 각 부서 총무 담당이 전달한다.")
IN6 = {
    "minutes": P6_MINUTES, "summary": "",
    "feedback": "", "grade": "",
    "tries": 0}
```

1단계: 상태와 평가 스키마 정의

```
class Verdict(BaseModel):
    grade: Literal["합격", "반려"]
    feedback: str = Field(description=D3)

class SummaryState(TypedDict):
    minutes: str      # 회의록 본문
    summary: str       # 한 줄 요약
    feedback: str      # 반려 사유
    grade: str         # 합격 / 반려
    tries: int         # 시도 횟수

grader = llm.with_structured_output(Verdict)
```



evaluator의 판정 형식. **Verdict**가 평가 결과의 스키마. 합격·반려 두 값 중 하나와 반려 사유 한 문장으로 고정됨.

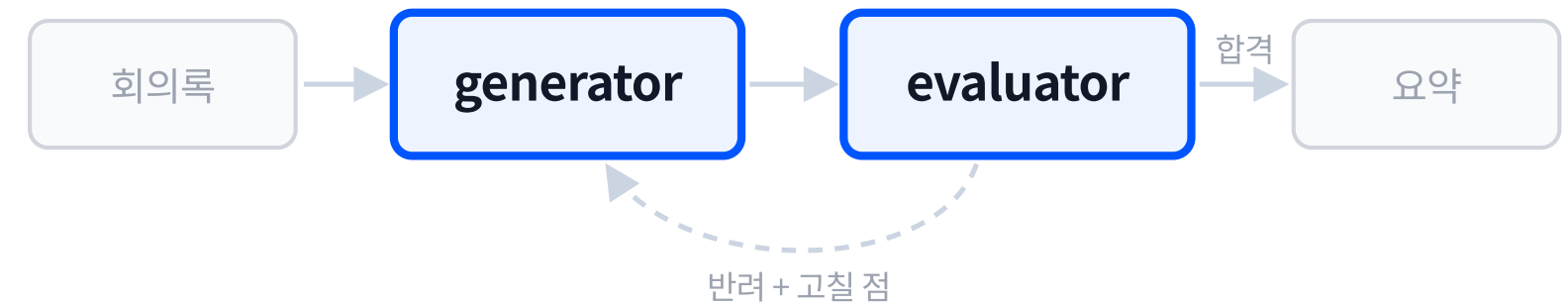
tries는 시도 횟수를 담는 상태 값. 종료를 판정할 때 이 값을 읽음.

with_structured_output으로 묶은 모델은 판정을 스키마대로 돌려줌. 답변 문자열을 뒤져 합격 여부를 찾아내지 않음.

2단계: 생성자 노드와 평가자 노드

```
def generator(s: SummaryState) -> dict:
    p, h = S_GEN1, s["minutes"]
    if s["feedback"]:      # 반려 사유는 입력 층에
        p = S_OPT
        h = "지적: " + s["feedback"] + "\n" + h
    r = llm.invoke([SystemMessage(p),
                    HumanMessage(h)])
    return {"summary": r.content.strip(),
            "tries": s["tries"] + 1}
```

```
def evaluator(s: SummaryState) -> dict:
    h = ("회의록: " + s["minutes"] +
         "\n요약: " + s["summary"])
    v = grader.invoke([SystemMessage(S_EVAL),
                       HumanMessage(h)])
    return {"grade": v.grade,
            "feedback": v.feedback}
```



generator 노드와 evaluator 노드. **generator**는 반려 사유가 상태에 들어 있으면 그것을 실은 프롬프트로, 비어 있으면 첫 프롬프트로 요약을 만듦. 한 노드가 첫 시도와 다시 쓰기를 함께 맡음.

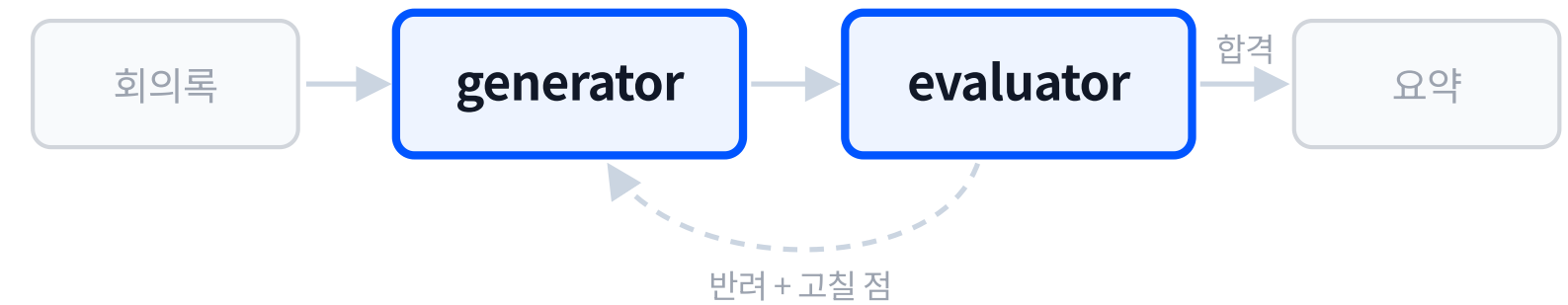
evaluator는 회의록 본문과 요약을 함께 넣어 판정을 받음. 평가자는 요약을 고치지 않고 판정과 사유만 돌려줌.

두 노드가 쓰는 상태 값이 나뉘어 있어 서로의 값을 덮지 않음.

3~4단계: 되돌림 판단과 그래프 연결

```
def route_summary(s: SummaryState) -> str:
    ok = s["grade"] == "합격"
    if ok or s["tries"] >= MAX_TRIES:
        return END      # 상한이 무한 루프를 막는다
    return "generator"

g = StateGraph(SummaryState)
g.add_node("generator", generator)
g.add_node("evaluator", evaluator)
g.add_edge(START, "generator")
g.add_edge("generator", "evaluator")
g.add_conditional_edges(
    "evaluator", route_summary,
    ["generator", END])
graph = g.compile()
out = graph.invoke(IN6)
print(out["tries"], out["grade"])
print(out["summary"])
```



evaluator에서 되돌아가는 화살표와 실행 부분.

route_summary가 종료 조건 자체. 합격이거나 시도 상한에 닿으면 END, 아니면 생성 노드로 되돌림.

MAX_TRIES 상한이 없으면 반려가 이어질 때 루프가 스스로 멈추지 못함.

평가자 위치에 사람(**interrupt()**)이 있으면 사람 승인 루프가 되고, LLM이 있으면 평가자-최적화자 패턴이 됨. 같은 위치에 다른 평가자가 있는 구성.

실행 결과

```
[generator] 진입 시도 1회차
  [요약 1회차] 회의실 예약을 다음 달 사내 포털로 이전한다.
[evaluator] 진입 -> grade='반려'
  [판정 사유] 요약에 이전 결정은 담겼지만, 이전 작업 담당자인
              총무팀과 사용법 안내 담당자인 각 부서 총무 담당이
              빠졌습니다.
[generator] 진입 시도 2회차
  [요약 2회차] 회의실 예약 시스템을 다음 달부터 사내 포털로
              이전하며, 총무팀이 이전 작업을 맡고 각 부서 총무
              담당이 사용법을 안내하기로 했다.
[evaluator] 진입 -> grade='합격'
  [판정 사유]
[최종 State] grade='합격' tries=2
```

Multi-Agent 설계/구현

실패 복구 전략

문제상황과 목표 / 판단표 / 준비 / 구현

학습 목표

- 멀티에이전트에서 생긴 실패를 「재시도로 고쳐질 실패」와 「재시도해도 무의미한 실패」로 구별할 수 있습니다.
- 각 실패에 대해 노드 재시도(RetryPolicy)·수퍼바이저 에스컬레이션(escalation)·사람 개입 중 무엇이 필요한지 근거와 함께 선택할 수 있습니다.

▶ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 수퍼바이저

패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

▶ 실패 복구 전략

서비스 설계와 리뷰

문제상황과 목표

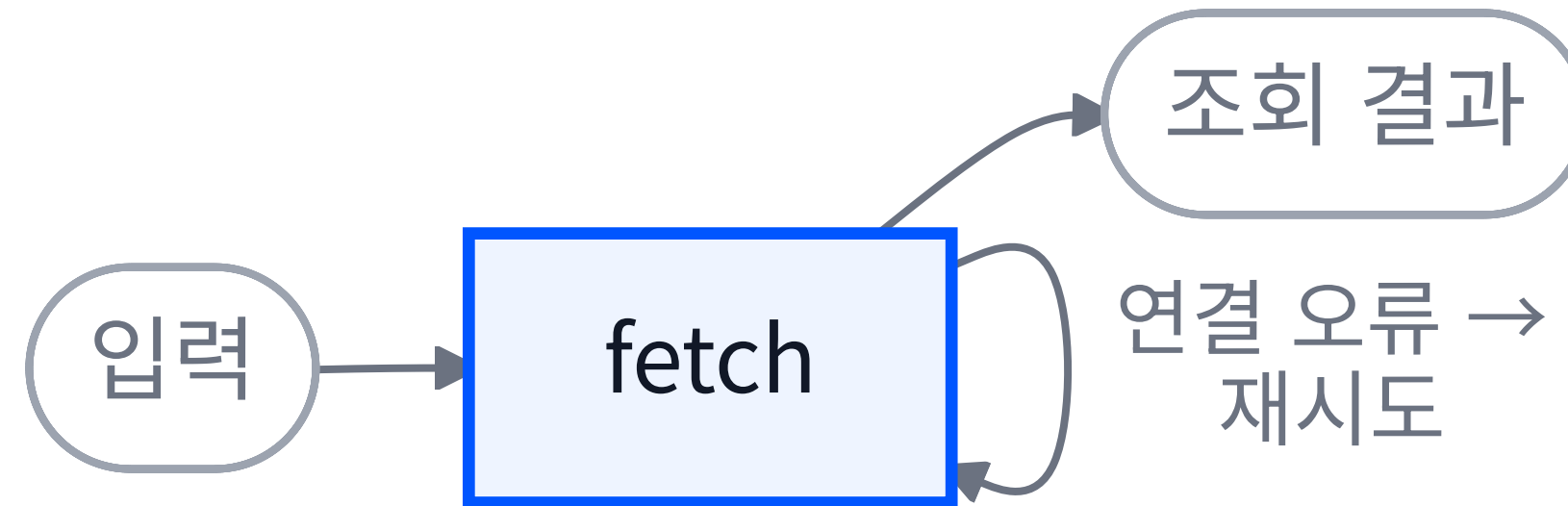
문제

물류팀 주니어 에이전트 개발자인데요, 외부 재고 조회가 가끔 실패하면 워크플로우가 그 자리에서 멈춰요. 그때마다 사람이 다시 눌러야 해요 ㅠㅠ

목표

재고 조회가 가끔 실패해도 워크플로우가 멈추지 않도록, 실패하면 정해진 횟수까지 스스로 다시 시도하게 만들어 사람이 최대한 덜 개입해도 되게 해봅시다.

각 단계별 요구사항



노드	무엇을 받아	어떻게	무엇을 넘긴다
fetch	재고 조회 요청을	연결 오류가 나면 정책대로 최대 3회까지 다시 시도해서	조회 결과와 성공한 시도 횟수를 넘긴다

실패 대응 판단표

이 표의 어느 장면에도 해당하지 않는 입력이 반드시 들어옵니다.

실패 장면	무엇이 실패했나	재시도로 고쳐지나	선택할 대응
조회 도구가 연결 오류로 죽음	일시적 장애	고쳐짐	<code>RetryPolicy</code> 재시도
분류 노드가 전담 노드를 잘못 고름	판단 자체가 틀림	안 고쳐짐	에스컬레이션(escalation)
워커 산출이 근거 없이 얇음	역할 수행의 품질	지시를 고쳐야 고쳐짐	에스컬레이션(escalation)
평가자가 같은 이유로 반려를 반복	종료 조건 미충족	안 고쳐짐	사람 개입 <code>interrupt()</code>
어느 실패 장면에도 해당하지 않는 입력	설계 밖의 입력	안 고쳐짐	사람 개입 <code>interrupt()</code>

같은 입력으로 다시 불렀을 때 결과가 달라질 수 있는 실패만 재시도로 고쳐짐. 판단이 틀린 실패는 몇 번을 다시 불러도 같은 결과가 나옴.

준비: 기본 코드

```
class FetchState(TypedDict):
    attempt: int
    result: str

_p8_calls = 0
IN8 = {"attempt": 0, "result": ""}
```



코드 항목	어디서 쓰나
FetchState	1단계 State: 시도 횟수와 조회 결과
_p8_calls	2단계 Node: 몇 번째 호출인지 세는 모듈 변수
IN8	4단계 실행: 초기 State

fetch 노드의 상태와 입력. `_p8_calls` 는 그래프 밖에 두어, 노드가 다시 불려도 값이 이어지도록 함. 재시도를 눈으로 세는 장치.

노드에 재시도 정책 부착

```
def flaky_fetch(s: FetchState) -> dict:
    global _p8_calls
    _p8_calls += 1
    if _p8_calls < 3:
        raise ConnectionError("조회 실패")
    return {"attempt": _p8_calls,
            "result": "재고 12개"}

policy = RetryPolicy(
    max_attempts=3, initial_interval=0.1,
    backoff_factor=1.0, jitter=False,
    retry_on=ConnectionError)
g = StateGraph(FetchState)
g.add_node("fetch", flaky_fetch,
           retry_policy=policy)
g.add_edge(START, "fetch")
g.add_edge("fetch", END)
print(g.compile().invoke(IN8))
```



fetch 노드와 재시도 정책. **RetryPolicy**는 "Configuration for retrying nodes." 속성은 **initial_interval**, **backoff_factor**, **max_interval**, **max_attempts**, **jitter**, **retry_on** 여섯 가지.

정책은 그래프가 아니라 **Node**에 붙음. **add_node**의 **retry_policy** 인자가 그 위치.

실행: **flaky_fetch** 1회차·2회차 **ConnectionError**, 3회차 성공. 최종 State는 attempt=3, result="재고 12개".

실행 결과

```
=== RetryPolicy: ConnectionError에 대해서만 0.1초 간격으로  
    최대 3번 재시도한다 ===  
[flaky_fetch] 진입 1회차  
    ConnectionError: [모의 장애] 1회차 조회 실패  
[flaky_fetch] 진입 2회차  
    ConnectionError: [모의 장애] 2회차 조회 실패  
[flaky_fetch] 진입 3회차  
[최종 State] attempt=3 result='재고 12개'
```

Multi-Agent 설계/구현

개발을 위한 기술 문서 묶음: 설계와 리뷰

문서 없는 위임 / 요구사항 / 아키텍처 다이어그램 / 결정 기록

학습 목표

- 만들 것을 목적 · 사용자 · 입출력 예시 · 도구 · 완료 기준으로 나눠 적고, 위임받는 쪽이 다시 물어야 할 것이 남았는지 판정할 수 있습니다.
- 아키텍처 다이어그램 한 장과 결정 기록 1~2쪽과 노드 계약 칸으로 설계를 문서화할 수 있습니다.

✓ 멀티에이전트 아키텍처

멀티에이전트가 필요한 순간

패턴 1 병렬화

패턴 2 라우팅

패턴 3 핸드오프

패턴 4 슈퍼바이저

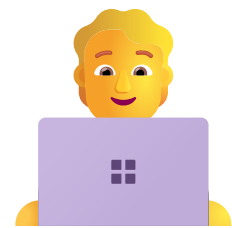
패턴 5 오케스트레이터-워커

패턴 6 평가자-최적화자

실패 복구 전략

▶ 서비스 설계와 리뷰

문서 없이 넘긴 설계



담당자

사내 문의를 유형별로 나눠 처리하는 그래프를 만들어 주세요.

만들었습니다. 문의를 세 유형으로 분류해 각 전담 노드로 보냅니다.

유형은 다섯이고, 분류가 애매하면 사람 승인을 받아야 합니다.

다섯 유형으로 고쳤습니다. 애매한 경우는 로그를 남기게 했습니다.

로그가 아니라 승인 대기입니다. 완료 기준은 무엇으로 잡으셨나요.

완료 기준은 전달받은 것이 없어 임의로 정했습니다.



코딩 에이전트

요구사항 문서: 목표에서 출발합니다

요구사항은 단순히 필요한 기능들의 목록이 아니라, '목표'를 달성하기 위해 필요한 기능들을 정리하는 것입니다. 반대로 말하면, 모든 요구사항에 대해서 "이 요구사항이 목표 달성에 어떤 기여를 하는지" 확인해야 합니다.

1. 사용자가 해결하고 싶은 문제와 사용 환경으로부터 목표와 제약을 명확히 한다.
2. 목표를 달성하기 위해 필요할 것 같은 기능, 제약 등을 일단 적어본다. 다이어그램이나 플로우 차트를 그려가며 상상해도 좋다. 왜 필요한 것 같은지 근거를 같이 적는다.
3. 각 요구사항이 목표 달성을 위해 정말 필요한 것인지 근거와 함께 판단하고, 불필요하면 지워낸다.
4. 각 요구사항이 프로젝트의 제약조건이나 보안사항을 어기지 않는지 확인하고 수정한다.
5. 요구사항이 명확하고 검증 가능한지 확인한다. 검증 방법을 떠올릴 수 없다면 다시 고민한다.
6. 남은 요구사항을 바탕으로 목표를 해결하기 위한 다이어그램/플로우 차트를 다시 그려본다.
7. 전체 흐름을 보면서 목표가 실제로 달성되는지, 모든 제약사항을 만족하는지 확인한다.
8. 다시 2번으로, 필요하다면 다시 1번으로. 필요한 요구사항은 빠지지 않고, 불필요한 요구사항은 남지 않을 때까지 반복한다.

아키텍처 다이어그램과 설계

1. 플로우차트, 혹은 아키텍처 다이어그램의 모든 단계에 대해서 '입력', '책임', '출력', '상태', '제약', '경계'이 명확해야 한다.

2. 공격하라.

- '공격A': 이 노드/연결을 삭제해도 목표가 달성되는가?
- '공격B': 이 노드/연결이 죽거나, 끊기거나, 느려지면 무슨일이 벌어지는가?
- '공격C': 이 연결이 왜곡되면 무슨일이 벌어지는가?
- '공격D': 이 제약 조건이 만족되지 못하면 무슨일이 벌어지는가?
- '공격E': 장애가 벌어졌을 때 탐지할 수 있나? 원인 추적이 되는가? 복구 가능한가?
- '공격F': 각 단계에서 범위 밖의 입력이 들어오면 무슨일이 벌어지는가?
- '공격G': 각 단계에서 많은 작업이 몰리면 무슨 일이 벌어지는가?
- '공격H': 각 단계가 제대로 수행되었다는 것을 어떻게 검증하나?

3. 문제가 발견되면, 아키텍처에서만 수정하려 하지마라. 다시 요구사항 분석단계로 돌아갈 각오를 하라. 요구사항 분석과 아키텍처 설계는 반복적으로 서로를 수정한다.

※ 책임: 어떤 역할과 기능을 맡아서 무엇을 보장해야 하는가?

※ 상태: 현재 어떤 상황에 있으며, 이후의 동작을 결정하기 위해 무엇을 기억하고 있는가?

※ 경계: 각 단계의 책임과 권한은 어디까지인가? 외부와 무엇을 주고받으며, 외부에서 들어온 정보나 요청을 어디까지 그대로 믿을 수 있는가?

결정 기록(ADR)

1. 문제 / 맥락

- 무엇을 결정해야 하는가?
- 어떤 요구사항이나 제약 때문에 이 결정이 필요한가?

2. 선택지

- 어떤 대안들이 있었는가?
- 최소한 실제로 고민했던 대안은 남긴다.

3. 판단 기준

- 무엇을 중요하게 봤는가?
- 성능, 비용, 개발시간, 복잡도, 보안, 유지보수성 등.

4. 결정

- 무엇을 선택했는가?

5. 근거

- 왜 이것을 선택했는가?
- 특히 왜 다른 선택지를 버렸는가?

6. 결과 / 대가

- 이 결정으로 무엇이 좋아지는가?
- 무엇을 포기하거나 감수하는가?

7. 재검토 조건

- 어떤 조건이 바뀌면 이 결정을 재검토해야 하는가?

실습합시다

참고: 이 강의에서 다루지 않은 것

이름	무엇인가	공식 문서
Domain Crew 자산 분리	에이전트 묶음(Crew)을 업무 영역별로 나누어 두는 구성	CrewAI, Crews
Multi-Crew 흐름	여러 Crew를 한 서비스 흐름으로 잇는 구성	CrewAI, Processes
완전 연결형 핸드오프	에이전트마다 핸드오프 도구를 갖춰 어느 에이전트에게든 넘기는 구성	LangChain, Multi-agent 「Handoffs」
서브그래프판 핸드오프	부모 그래프의 다른 에이전트로 넘기는 구성	LangChain, Multi-agent 「Handoffs」 § Multiple agent subgraphs
퍼블릭 배포·호출 검증	서비스를 외부 호출이 가능한 곳에 올리고 호출을 확인	LangChain Academy, Module 6 Deployment
트레이스 병목 최적화	트레이스에서 최다 지연·비용 구간을 찾아 개선	LangChain Academy, LangSmith Studio

강의 요약

- ✓ 에이전트를 나누는 기준: 도구를 더해서 되는 일은 나누지 않고, 한 프롬프트 안에서 역할이 서로를 밟거나, 하위 작업을 나란히 돌려야 하거나, 만드는 일과 판정하는 일을 따로 두어야 할 때 에이전트를 나눔
- ✓ 병렬화(Parallelization)와 오케스트레이터-워커(Orchestrator-Workers): 팬아웃할 노드 수를 설계 시점에 그림으로 못 박는 것이 병렬화이고, 입력을 보고 팬아웃할 수를 정하는 것이 오케스트레이터-워커
- ✓ 라우팅(Router)과 핸드오프(Handoff): 중앙이 입력을 분류해 전담 노드로 보내는 것이 라우팅이고, 도구가 상태 변수를 바꿔 에이전트가 서로 넘어가는 것이 핸드오프
- ✓ 슈퍼바이저(Supervisor): 중앙 에이전트가 서브에이전트를 도구처럼 부르고 모든 이동이 슈퍼바이저를 거침. 서브에이전트의 구현형이 서브그래프
- ✓ 평가자-최적화자(Evaluator-Optimizer): 생성 노드와 평가 노드가 되돌림 루프를 돌고, 종료 조건은 평가 기준과 반복 상한
- ✓ 실패 복구: 실패 대응은 재시도 · 에스컬레이션 · 사람 개입으로 나뉘고, 실패마다 근거를 들어 대응을 고름
- ✓ 설계 문서: 요구사항에서 출발해, 무엇을 받아 어떤 기능으로 무엇을 내는지를 다이어그램으로 정하고, 결정의 근거를 기록으로 남김

부록: 실습 파일 머리(모든 코드의 공통 импорт)

```
import operator
from typing import (Annotated, Literal,
                    TypedDict)

from langchain.chat_models import (
    init_chat_model)
from langchain.messages import (AIMessage,
    HumanMessage, SystemMessage, ToolMessage)
from langchain.tools import ToolRuntime
from langchain_core.tools import tool
```

각 패턴의 코드는 이 머리 아래에 붙여 한 파일로 실행합니다. 키는 환경변수 `OPENAI_API_KEY` 로 준비합니다.

```
from langgraph.graph import (END, START,
                             StateGraph)
from langgraph.graph.message import (
    add_messages)
from langgraph.prebuilt import ToolNode
from langgraph.types import (Command,
    RetryPolicy, Send)
from pydantic import BaseModel, Field

llm = init_chat_model(
    "openai/gpt-5.6-luna",
    model_provider="litellm")
```

상수와 시스템 프롬프트, 입력 예시는 챕터마다 「준비」 장에 있음.

